

Change Log

28th Aug 2025

We have updated the private user stream(socket) code in python for both balance updates and order updates.

24th July 2025

We have introduced Options trading via API is currently live for a select group of users.

13th Dec 2024

We have added private user stream(socket) for balance updates. This can be used to get real time balance updates of both the wallet and the portfolio whenever there is an update to any existing order.

25th July 2024

We have updated Trading Fee api response to include discount and fee after discount.

23rd July 2024

New Endpoints:

- Trading Fee `GET /trade/api/v2/tradingFee`

We have introduced an endpoint that will give you the fee applicable to you for an exchange.

2nd July 2024

- To enhance more security and better user experience we have changed the way the signature is getting generated and added epoch time (in millisecond) in the header and the same epoch time needs to be used for signature generation. To use any of the above APIs you have to mandatorily move to the new security construct. The older construct will have different problems in terms of security and precision matching. Currently the existing way of signature generation will work, but the same will be **phased out by 15th of August**.

New Endpoints:

- Trade Info `GET /trade/api/v2/tradeInfo`

We have introduced an endpoint that will give you the basic details required to start your trading. This includes Exchange Precision, Min Max for the active coins on our platform.

- TDS `GET /trade/api/v2/tds`

We have introduced an endpoint that will give you the TDS accumulated for your orders. (Note: TDS from C2C orders will come with a delay of 1 day)

- Server Time `GET /trade/api/v2/time`

24th May 2024

We have added private user stream(socket) for order updates. This can be used to get real time order updates on the orders that are placed during API trading across different exchanges.

9th February 2024

- We have launched a new exchange **c2c2** for USDT pairs.
- We have changed the way the sockets are getting connected, the handshake path is changed from *pro/realtime-rates-socket* to *pro/realtime-rates-socket/spot/{parameter}*. This is backward compatible with the older version. The older version will be moved out by 1st of April, 2024.

16th January 2024

- The signature generation logic for **Java** is updated for better code execution by removing issues of precision, but the older one keeps working.

26th December 2023

- Request limits for create order, portfolio, open order and close order APIs have been updated.

26th December 2023

- Request limits for create order, portfolio, open order and close order APIs have been updated.

10th November 2023

- We have introduced WebSocket for candlestick data that fetches data on realtime at every 1 min interval.
- We have optimised closed order for receiving more data and thereby reducing the number of calls that need to be made. Default value of **count** in the request has been changed to 500 i.e. we can now get at max 500 latest data entries on a single call of **closed order** . We have removed **total_orders_count** in the response json of the **closed order** and **open order** API.

6th November 2023

In Portfolio API, we have removed blocked_balance_deposit, blocked_balance_withdraw, blocked_balance_stake, blocked_balance_vault response fields.

25th September 2023

We have introduced two new status "CANCELLATION_RAISED" and "EXPIRATION_RAISED". It is an intermediate state.

CANCELLATION_RAISED means that we have asked the exchange to cancel the order but it is not cancelled on the exchange yet.

EXPIRATION_RAISED means that the order is getting expired from the broker end. During this period order cancellation is not allowed.

New Endpoints:

- Order Detail `GET /trade/api/v2/order`
- Enabled Coins `GET /trade/api/v2/coins`

- All response will be consistent in terms of contract but will contain **C2C** pairs of USDT.
- Parameters "S", "a", "b" are removed from both Trade API and trade socket response.
- "baseAsset", "quoteAsset" are removed from both ticker API and ticker socket response.

4th August 2023

We have improved our V1 APIs:

- Improved security in V2 APIs.
- API responses have become consistent.

python

java

New Endpoints:

- Validate Keys `GET /trade/api/v2/validate/keys`
- Ping `GET /trade/api/v2/ping`
- Create Order `POST /trade/api/v2/order`
- Cancel Order `DELETE /trade/api/v2/order`
- Open Order `GET /trade/api/v2/orders`
- Close Order `GET /trade/api/v2/orders`
- Portfolio `GET /trade/api/v2/user/portfolio`
- Exchange Precision `POST /trade/api/v2/exchangePrecision`
- 24hr all pairs `GET /trade/api/v2/24hr/all-pairs/ticker`
- 24hr specific pair `GET /trade/api/v2/24hr/ticker`
- Depth `GET /trade/api/v2/depth`
- Trade `GET /trade/api/v2/trades`

Streamline Your Trading Experience

Discover the power of CoinSwitch PRO API trading for spot and futures exchanges! This comprehensive guide will help you leverage our API to automate your trades, access real-time crypto rates, and create seamless integrations for your trading experience.

Our REST API is structured with user-friendly endpoints, giving you easy access to essential market data and exchange status information. We also provide private authenticated endpoints for advanced features such as trading and user-specific data. These endpoint requests need to be signed, for enhanced security and protection for your transactions.

Setting up an API Key

To fully utilize certain endpoints, you'll need an API key and a secret Key. You can generate this key pair on your [CoinSwitch PRO Profile](#)

In case you face any issues, please email your API key generation request to us at api@coinswitch.co. This step ensures an added layer of protection for your account and data.

Remember: Never share your API key or secret key with anyone. If your API keys have been unintentionally shared, we urge you to promptly delete them and generate a fresh key to maintain the highest level of security.

You can only have one API and one secret key active at a time.

Information about our APIs

At CoinSwitch PRO, our base endpoint for API calls is <https://coinswitch.co>. All our endpoints for Spot and Futures provide responses in the form of JSON objects or arrays, ensuring easy integration with your applications.

To maintain consistency, all time and timestamp related fields within our API follow a standardized format of milliseconds. This precision allows for accurate calculations and seamless data synchronization.

In each of the APIs we need the following headers :

Header	Value
Content-Type	application/json
X-AUTH-SIGNATURE	Singature that you need to generate at your end
X-AUTH-APIKEY	API key provided by Coinswitch

How to Setup

Begin API Trading on CoinSwitch PRO Spot and Futures by following the mentioned set-up steps based on your preferred client language.

Python client

Setup With PyCharm

To set up a Python file in PyCharm, follow these step-by-step instructions:

Install PyCharm

Download and install the PyCharm community version IDE from the JetBrains website <https://www.jetbrains.com/edu-products/download/other-PCE.html>). Choose the appropriate version for your operating system and install Python 3 as well.

Create a new project

Launch PyCharm and click on **Create New Project** on the welcome screen.

If you already have a project open, you can also go to **File -> New Project** to create a new one.

Configure project settings

In the **New Project** dialog, choose the location where you want to store your project files and give your project a name.

You can also specify the Python interpreter you want to use for this project. If you have a specific interpreter installed, select it; otherwise, you can choose the default option.

Create a new Python file

Once the project is created, you'll see the project structure on the left-hand side of the PyCharm window.

Right-click on the directory where you want to create your Python file (e.g., the project root directory) and select **New -> Python File**.

Give the file a meaningful name and press **Enter**

coinswitchPRO

Q Search

Change Log

Streamline Your Trading Expe...

Setting up an API Key

Information about our APIs

How to Setup

Get Set Go

Signature Generation

Keys Validation

Ping Testing

Introduction to C2C Trading

SPOT Trading APIs

Error Scenarios

WebSockets (Market Data)

Client Set Up for Websockets

WebSockets (User data)

Futures Trading APIs

Futures Public APIs

Futures WebSockets

Options Trading APIs

HFT (High-Frequency Trading)

HFT APIs

HFT Sockets

Terms and Conditions (T&Cs)

Contact us

Sign Up for a Developer Key

Documentation Powered by Slate

Write your Python code or paste api trading python file

The newly created Python file will open in the editor window. You can start writing your Python code here or paste the API trading python file that is shared with you over the email. PyCharm provides features like syntax highlighting, code completion, and debugging assistance to help you during development.

Install dependency

Open PyCharm Terminal and run below command

pip install cryptography==40.0.2

pip install requests==2.27.1

pip install python-socketio==5.5.1

pip install websocket-client==1.2.3

Run the Python file

To run your Python file, you can right-click anywhere in the editor window and select **Run** from the context menu.

Alternatively, you can use the keyboard shortcut (**Ctrl+Shift+F10** on Windows/Linux or **Shift+Control+R** on macOS) to run the current Python file.

View the output

After running the Python file, you'll see the output in the **Run** tool window at the bottom of the PyCharm interface. If your program generates any output, it will be displayed here.

That's it! You have successfully set up the environment for API Trading .

Note: *Please ensure you install Python 3.9 and the following libraries to their latest versions -cryptography, requests via your terminal in order to successfully run your scripts in your local machine without any virtual environment.*

Java Client

Here's a guide to help you get started with setting up a Java project with IntelliJ and Gradle:

Install IntelliJ IDEA

Download and install the latest version of IntelliJ IDEA from the official JetBrains website <https://www.jetbrains.com/idea/>.

Install Gradle

Download and install Gradle by following the instructions on the official Gradle website <https://gradle.org/install/>.

Open Shared API Trading Gradle project in IntelliJ

Launch IntelliJ IDEA and select **Open Project** on the welcome screen. Choose **Gradle** on the left panel and click **Next** Select **Java** as the project type and click **Next** Specify the project name and location, then click **Finish**

Configure Gradle in IntelliJ

After creating the project, IntelliJ will prompt you to configure Gradle. Choose **Use auto-import** to automatically synchronize Gradle changes. Select the **Gradle JVM** (Java Virtual Machine) you want to use for your project.

Configure project structure

In the Project view on the left side of IntelliJ, navigate to the project folder. Right-click on the **src** folder and you will see two files: the first one is the main java file and second one is the config file Specify the class name and click **OK**

Configure dependencies with Gradle

Open the `build.gradle` file in the project root directory. Inside the `dependencies` block, specify the libraries and dependencies you want to include in your project. For example, to include the Apache Commons Lang library, you can add the following line:

```
dependencies { implementation 'org.apache.commons:commons-lang3:3.12.0' }
```

Build and run the project

Click on the **Gradle** tab on the right side of the IntelliJ window. Expand the project tree and navigate to the **Tasks** section. Double-click on **build** to build the project. Once the build is successful, you can run the project by right-clicking on the Java class file and selecting **Run**.

Congratulations! You have set up the API Trading Java project with IntelliJ using Gradle as the build tool.

Get Set Go

Add the received `secret_key` and `api_key` (keys shared over email or have been generated from Coinswitch Pro UI) in your script and start using API trading.

Signature Generation

To access CoinSwitch APIs you need to generate signature and pass it as header in each and every API call. In the request sync the epoch time of your machine with the server time given by the following API.

Server Time

Use the following endpoint to get server time. This server time will be used to validate the request epoch time passed in the header. It is recommended that the epoch time send should be less than 5000ms, but if its more than 60000ms the request will fail.

python

java

```
import json
import requests

payload={}

url = "https://coinswitch.co/trade/api/v2/time"
```




coinswitchPRO

Q Search

Change Log

Streamline Your Trading Expe...

Setting up an API Key

Information about our APIs

How to Setup

Get Set Go

Signature Generation

Keys Validation

Ping Testing

Introduction to C2C Trading

SPOT Trading APIs

Error Scenarios

WebSockets (Market Data)

Client Set Up for Websockets

WebSockets (User data)

Futures Trading APIs

Futures Public APIs

Futures WebSockets

Options Trading APIs

HFT (High-Frequency Trading)

HFT APIs

HFT Sockets

Terms and Conditions (T&Cs)

Contact us

Sign Up for a Developer Key

Documentation Powered by Slate

HTTP Request

METHOD GET

ENDPOINT https://coinswitch.co/trade/api/v2/time

Keys Validation

You can validate the keys provided by Coinswitch using the URL given below:

HTTP Request

METHOD GET

ENDPOINT https://coinswitch.co/trade/api/v2/validate/keys

pythonjava

```
'Content-Type': 'application/json',
}

response = requests.request("GET", url, headers=headers, json=payload)

The above command returns JSON structured like this:

{
  "serverTime": 1719905777483
}

from cryptography.hazmat.primitives.asymmetric import ed25519
from urllib.parse import urlparse, urlencode
import urllib
import json
import time

# example method  ['GET', 'POST', 'DELETE']

# params represent the query params we need to pass during GET call

def get_signature(method, endpoint, params, epoch_time):
    unquote_endpoint = endpoint
    if method == "GET" and len(params) != 0:
        endpoint += ('&', '?')[urlparse(endpoint).query == ''] + urlencode(params)
        unquote_endpoint = urllib.parse.unquote_plus(endpoint)

    signature_msg = method + unquote_endpoint + epoch_time

    request_string = bytes(signature_msg, 'utf-8')
    secret_key_bytes = bytes.fromhex(secret_key)
    secret_key = ed25519.Ed25519PrivateKey.from_private_bytes(secret_key_bytes)
    signature_bytes = secret_key.sign(request_string)
    signature = signature_bytes.hex()
    return signature

Lets take an example of a Get API:

from cryptography.hazmat.primitives.asymmetric import ed25519
from urllib.parse import urlparse, urlencode
import urllib
import json
import requests
import time

params = {
    "count": 20,
    "from_time": 1600261657954,
    "to_time": 1687261657954,
    "side": "sell",
    "symbols": "btc/inr,eth/inr",
    "exchanges": "coinswitchx,wazirx",
    "type": "limit",
    "open": True
}

endpoint = "/trade/api/v2/orders"
method = "GET"
payload = {}

secret_key = < secret_key > # provided by coinswitch
api_key = < api_key > # provided by coinswitch
epoch_time = str(int(time.time() * 1000))

unquote_endpoint = endpoint
if method == "GET" and len(params) != 0:
    endpoint += ('&', '?')[urlparse(endpoint).query == ''] + urlencode(params)
    unquote_endpoint = urllib.parse.unquote_plus(endpoint)

signature_msg = method + unquote_endpoint + epoch_time

request_string = bytes(signature_msg, 'utf-8')
secret_key_bytes = bytes.fromhex(secret_key)
secret_key = ed25519.Ed25519PrivateKey.from_private_bytes(secret_key_bytes)
signature_bytes = secret_key.sign(request_string)
signature = signature_bytes.hex()

url = "https://coinswitch.co" + endpoint

headers = {
    'Content-Type': 'application/json',
    'X-AUTH-SIGNATURE': signature,
    'X-AUTH-APIKEY': api_key,
    'X-AUTH-EPOCH': epoch_time
}

response = requests.request("GET", url, headers=headers, json=payload)

Lets take an example of a DELETE API:

from cryptography.hazmat.primitives.asymmetric import ed25519
from urllib.parse import urlparse, urlencode
import urllib
import json
import requests

payload = {
    "order_id": "698ed406-8ef5-4664-9779-f7978702a447"
}

endpoint = "/trade/api/v2/order"
method = "DELETE"
params = {}

secret_key = < secret_key > # provided by coinswitch
api_key = < api_key > # provided by coinswitch

signature_msg = method + endpoint + json.dumps(payload, separators=(',', ':'))

request_string = bytes(signature_msg, 'utf-8')
secret_key_bytes = bytes.fromhex(secret_key)
secret_key = ed25519.Ed25519PrivateKey.from_private_bytes(secret_key_bytes)
signature_bytes = secret_key.sign(request_string)
signature = signature_bytes.hex()

url = "https://coinswitch.co" + endpoint

headers = {
    'Content-Type': 'application/json',
    'X-AUTH-SIGNATURE': signature,
    'X-AUTH-APIKEY': api_key
}

response = requests.request("DELETE", url, headers=headers, json=payload)

from cryptography.hazmat.primitives.asymmetric import ed25519
from urllib.parse import urlparse, urlencode
import urllib
import json
import requests

params = {}
```

from cryptography.hazmat.primitives.asymmetric import ed25519

from urllib.parse import urlparse, urlencode

import urllib

import json

import requests

params = {}

Ping Testing

Use the code below to check if your ecosystem has been successfully connected to the CoinSwitch ecosystem.

HTTP Request

METHOD GET

ENDPOINT https://coinswitch.co/trade/api/v2/ping

Introduction to C2C Trading

CoinSwitch PRO brings you the opportunity to trade your favorite cryptocurrencies using other cryptocurrencies. We have currently launched USDT Pairs for our traders to trade in. An active API trader would be able to place and successfully execute a crypto-to-crypto order on CoinSwitch PRO on these pairs. You can refer to the below illustration for better understanding.

Here is an example of a **BUY/SELL C2C USDT Pair** trade respectively for better trader understanding –

[USDT - BTC] — C2C Trade BUY
User can now use his/her available USDT from their balances and trade and acquire BTC on CoinSwitch PRO

[BTC - USDT] — C2C Trade SELL
User can sell away his/her existing cryptocurrency to obtain corresponding amount of USDT on CoinSwitch PRO

SPOT Trading APIs

Order Status Description

Status	Description
OPEN	Order has not been executed at all
PARTIALLY_EXECUTED	Order has been partially executed
CANCELLED	User has cancelled the order
EXECUTED	Order has been executed
EXPIRED	Order has expired
DISCARDED	Order has not been processed (It has not been placed on the exchange)
CANCELLATION_RAISED	This is an intermeditiate state. We have requested the exchange to cancel the order. This status will come only when the order is cancelled on the exchange.
EXPIRATION_RAISED	This is an intermeditiate state. We have requested the exchange to expire the order.

Active Coins

Use the following endpoint to check all the active coins for an exchange:

HTTP Request

METHOD GET

python java

```
method = "GET"
payload = {}

secret_key = < secret_key > # provided by coinswitch
api_key = < api_key> # provided by coinswitch

unquote_endpoint = endpoint
if method == "GET" and len(params) != 0:
    endpoint += ('&', '?')[urlparse(endpoint).query == ''] + urlencode(params)
    unquote_endpoint = urllib.parse.unquote_plus(endpoint)

signature_msg = method + unquote_endpoint + json.dumps(payload, separators=(',', ':'))

request_string = bytes(signature_msg, 'utf-8')
secret_key_bytes = bytes.fromhex(secret_key)
secret_key = ed25519.Ed25519PrivateKey.from_private_bytes(secret_key_bytes)
signature_bytes = secret_key.sign(request_string)
signature = signature_bytes.hex()

url = "https://coinswitch.co" + endpoint

headers = {
    'Content-Type': 'application/json',
    'X-AUTH-SIGNATURE': signature,
    'X-AUTH-APIKEY': api_key
}

response = requests.request("GET", url, headers=headers, json=payload)
```

RESPONSE 200

```
{
  "message": "Valid Access"
}
```

RESPONSE 401

```
{
  "message": "Invalid Access"
}
```

```
import requests
import json

url = "https://coinswitch.co/trade/api/v2/ping"
payload = {}

headers = {
    'Content-Type': 'application/json',
    'X-AUTH-SIGNATURE': <api-signature>,
    'X-AUTH-APIKEY': <api-key>
}

response = requests.request("GET", url, headers=headers, json=payload)
```

Response

```
{
  "message": "OK"
}
```

```
import requests
import json
from urllib.parse import urlparse, urlencode

params = {
    "exchange": "wazirx",
}
```

ENDPOINT `https://coinswitch.co/trade/api/v2/coins`

Request Parameters

Parameter	Type	Mandatory	Description
exchange	string	YES	Allowed values: "coinswitchx"/"wazirx"/"c2c1"/ "c2c2" (case insensitive)

✔ Remember — Use valid API key and signature!

Exchange Precision

Use the following endpoint to check precision coin and exchange wise:

HTTP Request

METHOD `POST`

ENDPOINT `https://coinswitch.co/trade/api/v2/exchangePrecision`

Request Parameters

Parameter	Type	Mandatory	Description
exchange	string	Yes	(case insensitive) Allowed values: "coinswitchx"/ "wazirx" / "c2c1"/ "c2c2"
symbol	string	No	(case insensitive) Example: "BTC/INR" "SHIB/INR" "BTC/USDT"

✔ Remember — Use valid API key and signature!

Trade Info

Use the following endpoint to check trading info:

HTTP Request

METHOD `GET`

ENDPOINT `https://coinswitch.co/trade/api/v2/tradeInfo`

Request Parameters

Parameter	Type	Mandatory	Description
exchange	string	Yes	(case insensitive) Allowed values: "coinswitchx"/ "wazirx" / "c2c1"/ "c2c2"
symbol	string	No	(case insensitive) Example: "BTC/INR" "SHIB/INR" "BTC/USDT"

✔ Remember — Use valid API key

python java

```
endpoint = "/trade/api/v2/coins"

endpoint += ('&', '?')[urlparse(endpoint).query == ''] + urlencode(params)

url = "https://coinswitch.co" + endpoint

headers = {
    'Content-Type': 'application/json',
    'X-AUTH-SIGNATURE': <api-signature>,
    'X-AUTH-APIKEY': <api-key>
}

response = requests.request("GET", url, headers=headers, json=payload)
```

The above command returns JSON structured like this:

```
Response: 200
{
  "data": {
    "wazirx": [
      "BAT/INR",
      "BNB/INR",
      "CHR/INR",
      "CRV/INR",
      "ETH/INR",
      "FTM/INR",
      "LRC/INR",
      "LTC/INR",
      "MKR/INR",
      "OGN/INR",
      "OMG/INR",
      "REQ/INR",
      "SXP/INR",
      "TRX/INR",
      "UMA/INR",
      "UNI/INR",
      "XLM/INR",
      "XRP/INR",
      "YFI/INR",
      "ZRX/INR",
      "BICO/INR",
      "COMP/INR",
      "COTI/INR",
      "DOGE/INR",
      "GALA/INR",
      "IOST/INR",
      "PEPE/INR",
      "SAND/INR",
      "USDT/INR",
      "YFII/INR",
      "1INCH/INR",
      "ALICE/INR",
      "JASMY/INR",
      "MATIC/INR"
    ]
  }
}
```

```
import requests
import json
from urllib.parse import urlparse, urlencode

url = "https://coinswitch.co/trade/api/v2/exchangePrecision"

payload = {
    "exchange": "coinswitchx",
    "symbol": "BTC/INR"
}

headers = {
    'Content-Type': 'application/json',
    'X-AUTH-SIGNATURE': <api-signature>,
    'X-AUTH-APIKEY': <api-key>
}

response = requests.request("POST", url, headers=headers, json=payload)
```

The above command returns JSON structured like this:

```
{
  "data": {
    "coinswitchx": {
      "BTC/INR": {
        "base": 5,
        "quote": 2,
        "limit": 0
      }
    }
  }
}
```

```
import requests
import json

url = "https://coinswitch.co/trade/api/v2/tradeInfo"

params = {
    "exchange": "coinswitchx",
    "symbol": "BTC/INR"
}

headers = {
    'Content-Type': 'application/json',
    'X-AUTH-SIGNATURE': <api-signature>,
    'X-AUTH-APIKEY': <api-key>
}

response = requests.request("GET", url, headers=headers, params=params)
```

The above command returns JSON structured like this:

```
{
  "data": {
    "coinswitchx": {
      "BTC/INR": {
        "quote": {
          "min": "150",
          "max": "2500000"
        },
        "precision": {
          "base": 6,
          "quote": 2,
          "limit": 0
        }
      }
    }
  }
}
```


Trading Fee

Use the following endpoint to check the trading fee applicable to you for an exchange:

HTTP Request

METHOD `GET`

ENDPOINT `https://coinswitch.co/trade/api/v2/tradingFee`

Request Parameters

Parameter	Type	Mandatory	Description
exchange	string	Yes	(case insensitive) Allowed values: "coinswitchx"/ "wazirx" / "c2c1"/ "c2c2"

Remember — Use valid API key and Signature

Create Order

Use the following endpoint to place a new order on the exchange:

HTTP Request

METHOD `POST`

ENDPOINT `https://coinswitch.co/trade/api/v2/order`

REQUEST LIMIT: 100 requests per 10 second

Request Parameters

Parameter	Type	Mandatory	Description
side	string	Yes	Allowed values: "buy"/"sell" (case insensitive)
symbol	string	Yes	Should be in the format <i>base_currency/quote_currency</i> (case insensitive). The quote currency can be USDT/BTC/INR.
type	string	Yes	Order type , currently we support only only LIMIT order type
price	float	Yes	Price at which you want to place the order

pythonjava

```
import requests
import json

url = "https://coinswitch.co/trade/api/v2/tradingFee"

params = {
    "exchange": "coinswitchx",
}

headers = {
    'Content-Type': 'application/json',
    'X-AUTH-APIKEY': <api-key>
    'X-AUTH-SIGNATURE': <signature>
}

response = requests.request("GET", url, headers=headers, params=params)
```

The above command returns JSON structured like this:

```
{
  "data": {
    "coinswitchx": {
      "AVAX": {
        "maker_fee": 0.0009,
        "taker_fee": 0.0009,
        "maker_discount_percentage": 100,
        "taker_discount_percentage": 100,
        "maker_fee_after_discount": 0,
        "taker_fee_after_discount": 0,
        "timestamp": 1721909805
      },
      "AXP": {
        "maker_fee": 0.0009,
        "taker_fee": 0.0009,
        "maker_discount_percentage": 100,
        "taker_discount_percentage": 100,
        "maker_fee_after_discount": 0,
        "taker_fee_after_discount": 0,
        "timestamp": 1721909805
      },
      "AXS": {
        "maker_fee": 0.0009,
        "taker_fee": 0.0009,
        "maker_discount_percentage": 100,
        "taker_discount_percentage": 100,
        "maker_fee_after_discount": 0,
        "taker_fee_after_discount": 0,
        "timestamp": 1721909805
      },
      "ETH": {
        "maker_fee": 0.0009,
        "taker_fee": 0.0009,
        "maker_discount_percentage": 100,
        "taker_discount_percentage": 100,
        "maker_fee_after_discount": 0,
        "taker_fee_after_discount": 0,
        "timestamp": 1721909805
      },
      "FIL": {
        "maker_fee": 0.0009,
        "taker_fee": 0.0009,
        "maker_discount_percentage": 100,
        "taker_discount_percentage": 100,
        "maker_fee_after_discount": 0,
        "taker_fee_after_discount": 0,
        "timestamp": 1721909805
      },
      "PENDLE": {
        "maker_fee": 0.0009,
        "taker_fee": 0.0009,
        "maker_discount_percentage": 100,
        "taker_discount_percentage": 100,
        "maker_fee_after_discount": 0,
        "taker_fee_after_discount": 0,
        "timestamp": 1721909805
      },
      "SHIB": {
        "maker_fee": 0.0009,
        "taker_fee": 0.0009,
        "maker_discount_percentage": 100,
        "taker_discount_percentage": 100,
        "maker_fee_after_discount": 0,
        "taker_fee_after_discount": 0,
        "timestamp": 1721909805
      },
      "USDT": {
        "maker_fee": 0.0009,
        "taker_fee": 0.0009,
        "maker_discount_percentage": 100,
        "taker_discount_percentage": 100,
        "maker_fee_after_discount": 0,
        "taker_fee_after_discount": 0,
        "timestamp": 1721909805
      },
      "YFI": {
        "maker_fee": 0.0009,
        "taker_fee": 0.0009,
        "maker_discount_percentage": 100,
        "taker_discount_percentage": 100,
        "maker_fee_after_discount": 0,
        "taker_fee_after_discount": 0,
        "timestamp": 1721909805
      }
    }
  }
}
```

```
import requests
import json

url = "https://coinswitch.co/trade/api/v2/order"

payload = {
    "side":"sell",
    "symbol":"BTC/USDT",
    "type":"limit",
    "price":26000,
    "quantity":0.0009,
    "exchange":"c2c1"
}

headers = {
    'Content-Type': 'application/json',
    'X-AUTH-SIGNATURE': <api-signature>,
    'X-AUTH-APIKEY': <api-key>
}

response = requests.request("POST", url, headers=headers, json=payload)
```

The above command returns JSON structured like this:

```
RESPONSE 200

{
  "data":{
    "order_id":"927db2d9-643c-47a3-9c0f-78741ac95cf5",
    "symbol":"BTC/USDT",
```

quantity	float	Yes	Base quantity that you want to buy or sell
exchange	string	Yes	Allowed values: "coinswitchx"/"wazirx"/"c2c1"/"c2c2" (case insensitive)

✔ Remember — Use Valid API key and Signature!

Cancel Order

Use the following endpoint to cancel an order on the exchange:

HTTP Request

METHODDELETE

ENDPOINThttps://coinswitch.co/trade/api/v2/order

Request Parameters

Parameter	Type	Mandatory	Description
order_id	string	Yes	order_id that needs to be cancelled

✔ Remember — Use valid API key and signature!

Get Order

Use the following endpoint to check order details for a particular order:

HTTP Request

METHODGET

ENDPOINThttps://coinswitch.co/trade/api/v2/order

Request Parameters

Parameter	Type	Mandatory	Description
order_id	string	YES	order_id of the order that you want to fetch

✔ Remember — Use valid API key and signature!

python

java

```
"orig_qty":0.0009, # quantity at which order was placed
"executed_qty":0, # quantity that got executed
"status":"OPEN",
"side":"SELL",
"exchange":"c2c1",
"order_source":"API_TRADING",
"created_time":1689661638000, # time stamp in milli seconds
"updated_time":1689661639000 # time stamp in milli seconds
}
}

RESPONSE: 422
{
  "message": "Input price must be of type number"
}

RESPONSE: 400
{
  "message": "Amount is more than available balance"
}

RESPONSE: 401
{
  "message": "Invalid access"
}
```

```
import requests
import json

url = "https://coinswitch.co/trade/api/v2/order"

payload = {
  "order_id": "698ed406-8ef5-4664-9779-f7978702a447"
}

headers = {
  'Content-Type': 'application/json',
  'X-AUTH-SIGNATURE': <api-signature>,
  'X-AUTH-APIKEY': <api-key>
}

response = requests.request("DELETE", url, headers=headers, json=payload)
```

The above command returns JSON structured like this:

RESPONSE 200

```
{
  "data":{
    "order_id":"698ed406-8ef5-4664-9779-f7978702a447",
    "symbol":"BTC/INR",
    "price":2300000, # price at which Order was placed by the user
    "average_price":0, # price at which the order got executed
    "orig_qty":5e-05, # quantity at which order was placed
    "executed_qty":0, # quantity that got executed
    "status":"CANCELLED",
    "side":"SELL",
    "exchange":"coinswitchx",
    "order_source":"API_TRADING",
    "created_time":1689661638000, # time stamp in milli seconds
    "updated_time":1689661639000 # time stamp in milli seconds
  }
}
```

```
import requests
import json
from urllib.parse import urlparse, urlencode

params = {
  "order_id": "698ed406-8ef5-4664-9779-f7978702a447",
}
payload = {}

endpoint = "/trade/api/v2/order"

endpoint += ('&', '?')[urlparse(endpoint).query == ''] + urlencode(params)

url = "https://coinswitch.co" + endpoint

headers = {
  'Content-Type': 'application/json',
  'X-AUTH-SIGNATURE': <api-signature>,
  'X-AUTH-APIKEY': <api-key>
}

response = requests.request("GET", url, headers=headers, json=payload)
```

The above command returns JSON structured like this:

Response 200

```
{
  "data":{
    "order_id":"698ed406-8ef5-4664-9779-f7978702a447",
    "symbol":"BTC/USDT",
    "price":27000,
    "average_price":"0",
    "orig_qty":37.79055,
    "executed_qty":0,
    "status":"OPEN",
    "side":"SELL",
    "exchange":"c2c1",
    "order_source":"API_TRADING",
    "created_time":1695365983000,
    "updated_time":1695365983000
  }
}
```


Open Orders

Use the following endpoint to check open orders:

HTTP Request

METHOD `GET`

ENDPOINT `https://coinswitch.co/trade/api/v2/orders`

REQUEST LIMIT: `10000` requests per `10` second

Request Parameters

Parameter	Type	Mandatory	Description
count	string	No	Default value 25
from_time	string	No	Timestamp in milliseconds
to_time	string	No	Timestamp in miliseconds
side	string	No	Allowed values: (case insensitive) "buy" or "sell"
symbols	string	No	(case insensitive) Example: "BTC/INR" "SHIB/INR" " BTC/USDT " "btc/inr,eth/inr"
exchanges	string	No	Allowed values: (case insensitive) "coinswitchx,wazirx,c2c1, c2c2 " / "coinswitchx" / "wazirx" / "c2c1" / " c2c2 "
type	string	No	By default it will take Limit
open	bool	Yes	It should be true to get open orders

✔ Remember — Use valid API key and signature!

Closed Orders

Use the following endpoint to check closed orders:

HTTP Request

METHOD `GET`

ENDPOINT `https://coinswitch.co/trade/api/v2/orders`

REQUEST LIMIT: `10000` requests per `10` second

Request Parameters

Parameter	Type	Mandatory	Description
count	string	No	Default value 500
from_time	string	No	Timestamp in miliseconds
to_time	string	No	Timestamp in milliseconds
side	string	No	Allowed values: (case insensitive) "buy" or "sell"
symbols	string	No	(case insensitive) Example: "BTC/INR" "SHIB/INR" "BTC/USDT" "btc/inr,eth/inr"
exchanges	string	No	Allowed values: (case insensitive) "coinswitchx,wazirx,c2c1, c2c2 " / "coinswitchx" / "wazirx" / "c2c1" / " c2c2 "
type	string	No	Default value Limit
status	string	No	Order status
open	bool	No	Default value is False

Note : This API response would contain the latest 500 row data entries, for older records please reach out to us at api@coinswitch.co for support

✔ Remember — Use valid API key and signature!

pythonjava

```
import requests
import json
from urllib.parse import urlparse, urlencode
```

```
params = {
    "count": 2,
    "from_time": 1600261657954,
    "to_time": 1687261657954,
    "side": "sell",
    "symbols": "btc/inr,eth/inr",
    "exchanges": "coinswitchx,wazirx",
    "type": "limit",
    "open": True
}
```

```
payload = {}
```

```
endpoint = "/trade/api/v2/orders"
```

```
endpoint += ('&', '?')[urlparse(endpoint).query == ''] + urlencode(params)
```

```
url = "https://coinswitch.co" + endpoint
```

```
headers = {
    'Content-Type': 'application/json',
    'X-AUTH-SIGNATURE': <api-signature>,
    'X-AUTH-APIKEY': <api-key>
}
```

```
response = requests.request("GET", url, headers=headers, json=payload)
```

The above command returns JSON structured like this:

```
Response 200
{
  "data": {
    "orders": [
      {
        "order_id": "56013d06-7fe6-416f-9086-86902577ba99",
        "symbol": "SHIB/INR",
        "price": 0.000638,
        "average_price": 0.000957,
        "orig_qty": 187400,
        "executed_qty": 0,
        "status": "OPEN",
        "side": "SELL",
        "exchange": "coinswitchx",
        "order_source": "API_TRADING",
        "created_time": 1687173658000,
        "updated_time": 1687173705000
      },
      {
        "order_id": "87a1b145-84f4-4e8b-8199-0dd0fab2a5ec",
        "symbol": "SHIB/INR",
        "price": 0.000247,
        "average_price": 0,
        "orig_qty": 177400,
        "executed_qty": 167400,
        "status": "PARTIALLY_EXECUTED",
        "side": "SELL",
        "exchange": "coinswitchx",
        "order_source": "API_TRADING",
        "created_time": 1687171873000,
        "updated_time": 1687173633000
      }
    ]
  }
}
```

```
# If no orders are present
```

```
Response 200
{
  "data": {
    "orders": []
  }
}
```

```
import requests
import json
from urllib.parse import urlparse, urlencode
```

```
params = {
    "count": 3,
    "from_time": 1600261657954,
    "to_time": 1687261657954,
    "side": "sell",
    "symbols": "btc/inr,eth/inr",
    "exchanges": "coinswitchx,wazirx",
    "type": "limit",
    "status": "EXECUTED",
    "open": False
}
```

```
payload = {}
```

```
endpoint = "/trade/api/v2/orders"
```

```
endpoint += ('&', '?')[urlparse(endpoint).query == ''] + urlencode(params)
```

```
url = "https://coinswitch.co" + endpoint
```

```
headers = {
    'Content-Type': 'application/json',
    'X-AUTH-SIGNATURE': <api-signature>,
    'X-AUTH-APIKEY': <api-key>
}
```

```
response = requests.request("GET", url, headers=headers, json=payload)
```

The above command returns JSON structured like this:

```
Response 200
{
  "data": {
    "orders": [
      {
        "order_id": "56013d06-7fe6-416f-9086-86902577ba99",
        "symbol": "SHIB/INR",
        "price": 0.000638,
        "average_price": 0.000957,
        "orig_qty": 187400,
        "executed_qty": 0,
        "status": "CANCELLED",
        "side": "SELL",
        "exchange": "coinswitchx",
        "order_source": "API_TRADING",
        "created_time": 1687173658000,
        "updated_time": 1687173705000
      },
      {
        "order_id": "87a1b145-84f4-4e8b-8199-0dd0fab2a5ec",
        "symbol": "SHIB/INR",
        "price": 0.000247,
        "average_price": 0,
```

Portfolio

Use the following endpoint to check your portfolio:

HTTP Request

METHOD GET

ENDPOINT https://coinswitch.co/trade/api/v2/user/portfolio

REQUEST LIMIT: 5000 requests per 10 second

Remember — Use valid API key and signature!

TDS

Use the following endpoint to check your TDS:

(Note: TDS from C2C orders will come with a delay of 1 day)

HTTP Request

METHOD GET

ENDPOINT https://coinswitch.co/trade/api/v2/tds

REQUEST LIMIT: 25 requests per 10 second

Remember — Use valid API key and signature!

Trades

Use the following endpoint to get all trades history:

HTTP Request

METHOD GET

ENDPOINT https://coinswitch.co/trade/api/v2/trades

Request Parameters

Parameter	Type	Mandatory	Description
-----------	------	-----------	-------------

pythonjava

```
"status": "CANCELLED",
"side": "SELL",
"exchange": "coinswitchx",
"order_source": "API_TRADING",
"created_time": 1687171873000,
"updated_time": 1687173633000
},
{
  "order_id": "47a1b145-84f4-4e8b-8199-0dd0fab2a5ec",
  "symbol": "SHIB/INR",
  "price": 0.000247,
  "average_price": 0,
  "orig_qty": 177400,
  "executed_qty": 177400,
  "status": "EXECUTED",
  "side": "BUY",
  "exchange": "coinswitchx",
  "order_source": "API_TRADING",
  "created_time": 1687171873000,
  "updated_time": 1687173633000
}
]
}
}

# If no orders are present

Response 200
{
  "data": {
    "orders": []
  }
}
```

```
import requests
import json

url = "https://coinswitch.co/trade/api/v2/user/portfolio"

payload={}
headers = {
  'Content-Type': 'application/json',
  'X-AUTH-SIGNATURE': <api-signature>,
  'X-AUTH-APIKEY': <api-key>
}

response = requests.request("GET", url, headers=headers, json=payload)
```

The above command returns JSON structured like this:

Status Code 200

```
{
  "data":[
    {
      "currency":"ETH",
      "blocked_balance_order":"0.0019",
      "main_balance":"0.00008393",
      "buy_average_price":110882.72063601,
      "invested_value":220.40483679829757,
      "invested_value_excluding_fee":219.98355595139932,
      "current_value":324.10075659,
      "sell_rate":163363,
      "buy_rate":164531,
      "is_average_price_available":true,
      "name":"Ethereum",
      "is_delisted_coin":false
    },
    {
      "currency":"SHIB",
      "blocked_balance_order":"0",
      "main_balance":"99983.2566",
      "buy_average_price":0.00088449,
      "invested_value":88.498179914358,
      "invested_value_excluding_fee":88.434190630134,
      "current_value":67.38871494840001,
      "sell_rate":0.000674,
      "buy_rate":0.000674,
      "is_average_price_available":true,
      "name":"Shiba Inu",
      "is_delisted_coin":false
    },
    {
      "name":"Indian Rupee",
      "currency":"INR",
      "main_balance":802.33,
      "blocked_balance_order":0,
      "invested_value":308.9,
      "current_value":654.95
    }
  ]
}
```

```
import json
import requests

payload={}

url = "https://coinswitch.co/trade/api/v2/tds"

headers = {
  'Content-Type': 'application/json',
  'X-AUTH-SIGNATURE': <api-signature>,
  'X-AUTH-APIKEY': <api-key>,
}

response = requests.request("GET", url, headers=headers, json=payload)
```

The above command returns JSON structured like this:

```
{
  "data": {
    "total_tds_amount": 459.12,
    "financial_year": "2024-2025"
  }
}
```

```
import requests
import json

params = {
  "exchange": "wazirx",
  "symbol": "btc/inr"
}

endpoint = "/trade/api/v2/trades"

endpoint += ('&', '?')[urlparse(endpoint).query == ''] + urlencode(params)

url = "https://coinswitch.co" + endpoint
```

exchange	string	Yes	(case insensitive) Allowed values: "coinswitchx"/ "wazirx" / "c2c1"/ "c2c2"
symbol	string	Yes	(case insensitive) Example: "BTC/INR" "SHIB/INR" "BTC/USDT" . The quote currency can be USDT/BTC/INR.

✔ Remember — Use valid API key and signature!

Depth

Use the following endpoint to check orderbook:

HTTP Request

METHOD GET

ENDPOINT `https://coinswitch.co/trade/api/v2/depth`

Request Parameters

Parameter	Type	Mandatory	Description
exchange	string	Yes	(case insensitive) Allowed values: "coinswitchx" / "wazirx" / "c2c1"/ "c2c2"
symbol	string	Yes	(case insensitive) Example: "BTC/INR" "SHIB/INR" "BTC/USDT" .The quote currency can be USDT/BTC/INR.

✔ Remember — Use valid API key and signature!

Candles

Use the following endpoint to get candles data:

HTTP Request

METHOD GET

ENDPOINT `https://coinswitch.co/trade/api/v2/candles`

Request Parameters

Parameter	Type	Mandatory	Description
exchange	string	Yes	(case insensitive) Allowed values: "coinswitchx" / "wazirx" / "c2c1"/""*c2c2**"
symbol	string	Yes	(case insensitive) Example: "BTC/INR" "SHIB/INR" "BTC/USDT" . The quote currency can be USDT/BTC/INR.
interval	int	Yes	Duration of candlestick in minutes
start_time	long	Yes	Timestamp in milliseconds (Epoch format)
end_time	long	Yes	Timestamp in milliseconds (Epoch format)

Ticker 24hr All Pairs

Use the following endpoint to get 24hr ticker for all coins:

HTTP Request

METHOD GET

ENDPOINT `https://coinswitch.co/trade/api/v2/24hr/all-pairs/ticker`

python java

```
headers = {
    'Content-Type': 'application/json',
    'X-AUTH-SIGNATURE': <api-signature>,
    'X-AUTH-APIKEY': <api-key>
}

response = requests.request("GET", url, headers=headers, json=payload)
```

The above command returns JSON structured like this:

```
{
  "data": [
    {
      "E": 1686512630072, # Event time
      "m": false, # Is Buyer Maker
      "p": "5.8599", # Price
      "q": "42", # Quantity
      "s": "DOGE/INR", # Symbol
      "t": "f60742d8-2494-4d87-96e5-310ded278c15", # Trade ID
      "e": "coinswitchx" # Exchange
    }
  ]
}
```

```
import requests
import json
from urllib.parse import urlparse, urlencode
```

```
params = {
    "exchange": "wazirx",
    "symbol": "btc/inr"
}
```

```
endpoint = "/trade/api/v2/depth"
```

```
endpoint += ('&', '?')[urlparse(endpoint).query == ''] + urlencode(params)
```

```
url = "https://coinswitch.co" + endpoint
```

```
payload = {}
```

```
headers = {
    'Content-Type': 'application/json',
    'X-AUTH-SIGNATURE': <api-signature>,
    'X-AUTH-APIKEY': <api-key>
}
```

```
response = requests.request("GET", url, headers=headers, json=payload)
```

The above command returns JSON structured like this:

```
{
  "data": {
    "symbol": "DOGE/INR",
    "timestamp": 1686514361972,
    "bids": [
      [
        "5.82", //Price
        "4350" //Quantity
      ]
    ],
    "asks": [
      [
        "5.8664", //Price
        "1238" //Quantity
      ]
    ]
  }
}
```

```
import requests
from urllib.parse import urlparse, urlencode
```

```
params = {
    "end_time": "1662681600000",
    "start_time": "1647388800000",
    "symbol": "BTC/INR",
    "interval": "60",
    "exchange": "wazirx"
}
```

```
endpoint = "/trade/api/v2/candles"
```

```
endpoint += ('&', '?')[urlparse(endpoint).query == ''] + urlencode(params)
```

```
url = "https://coinswitch.co" + endpoint
payload = {}
```

```
headers = {
    'Content-Type': 'application/json',
    'X-AUTH-SIGNATURE': <api-signature>,
    'X-AUTH-APIKEY': <api-key>
}
```

```
response = requests.request("GET", url, headers=headers, json=payload)
```

The above command returns JSON structured like this:

```
{
  "result": [
    {
      "o": "1591001.000000000000", # Open
      "h": "1610988.000000000000", # High
      "l": "1588000.000000000000", # Low
      "c": "1591000.000000000000", # Close
      "interval": "60", # Candlestick duration
      "symbol": "BTC/INR", # case insensitive
      "close_time": "1662681600000", # Timestamp in milliseconds
      "volume": "0.304680000000", # trade volume
      "start_time": "1662678000000" # Timestamp in milliseconds
    }
  ],
}
```

```
import requests
from urllib.parse import urlparse, urlencode
```

```
params = {
    "exchange": "coinswitchx"
}
```

```
endpoint = "/trade/api/v2/24hr/all-pairs/ticker"
```


Parameter	Type	Mandatory	Description
exchange	string	Yes	(case insensitive) Allowed values "coinswitchx" / "wazirx" / "c2c1"/" c2c2 "

✔ Remember — Use valid API key and signature!

Ticker 24hr For Specific Coin

Use the following endpoint to get the 24hr ticker for specific coin:

HTTP Request

METHOD GET

ENDPOINT `https://coinswitch.co/trade/api/v2/24hr/ticker`

Request Parameters

Parameter	Type	Mandatory	Description
exchange	string	Yes	(case insensitive) Allowed values "coinswitchx" / "wazirx" / "c2c1"/" c2c2 "
symbol	string	Yes	(case insensitive) Example: "BTC/INR" "SHIB/INR" " BTC/USDT ".The quote currency can be USDT/BTC/INR.

✔ Remember — Use valid API key and signature!

Error Scenarios

HTTP Error Codes

- HTTP 4XX: Malformed request; the issue is on the sender's end.
- HTTP 429: Your request rate limit has been exceeded.
- HTTP 5XX: Internal error; issue on CoinSwitch end. It is important to NOT treat this as a failure operation; the execution status is UNKNOWN and could have been a success.

Custom Error Messages

- Status Code 422 { "message": "Input <field name> is missing" }
- Status Code 422 { "message": "The value you provided for the field i s not valid. Please check the documentation for valid values and try again." }
- Status Code 400 { "message": "Amount is more than available balance" }
- Status Code 401 { "message": "Invalid Access" }

pythonjava

```
url = "https://coinswitch.co" + endpoint
payload = {}

headers = {
    'Content-Type': 'application/json',
    'X-AUTH-SIGNATURE': <api-signature>,
    'X-AUTH-APIKEY': <api-key>
}

response = requests.request("GET", url, headers=headers, json=payload)

The above command returns JSON structured like this:

{
  "data": {
    "WAVES/INR": {
      "symbol": "WAVES/INR",
      "openPrice": "127.65",
      "lowPrice": "125.53",
      "highPrice": "129.88",
      "lastPrice": "125.56",
      "baseVolume": "1910.65",
      "quoteVolume": "240593.58",
      "percentageChange": "-1.63728946337642",
      "bidPrice": "125.56",
      "askPrice": "127.29",
      "at": 1687267004787
    },
    "DNT/INR": {
      "symbol": "DNT/INR",
      "openPrice": "0",
      "lowPrice": "0",
      "highPrice": "0",
      "lastPrice": "0",
      "baseVolume": "0",
      "quoteVolume": "0",
      "percentageChange": "0",
      "bidPrice": "154",
      "askPrice": "156",
      "at": 1687267004494
    }
  }
}
```

```
import requests
from urllib.parse import urlparse, urlencode

params = {
    "symbol": "BTC/INR",
    "exchange": "coinswitchx,wazirx"
}

endpoint = "/trade/api/v2/24hr/ticker"

endpoint += ('&', '?')[urlparse(endpoint).query == ''] + urlencode(params)

url = "https://coinswitch.co" + endpoint
payload = {}

headers = {
    'Content-Type': 'application/json',
    'X-AUTH-SIGNATURE': <api-signature>,
    'X-AUTH-APIKEY': <api-key>
}

response = requests.request("GET", url, headers=headers, json=payload)

The above command returns JSON structured like this:

{
  "data": {
    "WAVES/INR": {
      "symbol": "WAVES/INR",
      "openPrice": "127.65",
      "lowPrice": "125.53",
      "highPrice": "129.88",
      "lastPrice": "125.56",
      "baseVolume": "1910.65",
      "quoteVolume": "240593.58",
      "percentageChange": "-1.63728946337642",
      "bidPrice": "125.56",
      "askPrice": "127.29",
      "at": 1687267004787
    },
    "DNT/INR": {
      "symbol": "DNT/INR",
      "openPrice": "0",
      "lowPrice": "0",
      "highPrice": "0",
      "lastPrice": "0",
      "baseVolume": "0",
      "quoteVolume": "0",
      "percentageChange": "0",
      "bidPrice": "154",
      "askPrice": "156",
      "at": 1687267004494
    }
  }
}
```

WebSockets (Market Data)

How to connect ?

Socket Config	Value
Type	Socket.IO
Version	v4
Handshake Path	/pro/realtime-rates-socket/spot/{parameter}
Base URL	wss://ws.coinswitch.co/

Parameter	Type	Mandatory	Description
parameter	string	Yes	(case insensitive) Allowed values "coinswitchx" / "wazirx" / "c2c1"/" c2c2 "

Namespace

Exchange	Namespace
Wazirx	baseurl + /wazirx
CoinswitchX	baseurl + /coinswitchx
C2C1	baseurl + /c2c1
C2C2	baseurl + /c2c2

Client Set Up for Websockets

Order Book

Whenever a new order is placed or a change happens in order book/market depth, web socket will send an update.

Event Name = "FETCH_ORDER_BOOK_CS_PRO"

Candlestick

The socket will share the candlestick data last 1 min if trades are executed on given pair.

Event Name = "FETCH_CANDLESTICK_CS_PRO"

Trades

Whenever a trade gets executed, the web socket will send an update.

Event Name = "FETCH_TRADES_CS_PRO"

python

java

PRO Tip: To connect with a particular exchange, we have to provide namespace as we use multiplexing.

```
import socketio
sio = socketio.Client()

pair = "BTC,INR"
sio.connect(url=base_url, namespaces=[namespace], transports='websocket',
            socketio_path='/pro/realtime-rates-socket/spot', wait=True, wait_timeout=10)

subscribe_data = {
    'event': 'subscribe',
    'pair': pair
}

sio.emit(FETCH_ORDER_BOOK_CS_PRO, subscribe_data, namespace=namespace)
```

```
RESPONSE

{
  "s": "BTC,INR",
  "timestamp": 1673255767122,
  "bids": [
    [
      "1471201.000000000000", //Price
      "0.005870" //Quantity
    ],
    [
      "1470201.000000000000", //Price
      "0.046770" //Quantity
    ]
  ],
  "asks": [
    [
      "1484400.000000000000",
      "0.022109"
    ],
    [
      "1484500.000000000000",
      "0.000390"
    ]
  ]
}
```

```
import socketio
sio = socketio.Client()
pair = "BTC,INR"
sio.connect(url=base_url, namespaces=[namespace], transports='websocket',
            socketio_path='/pro/realtime-rates-socket/spot', wait=True, wait_timeout=10)

subscribe_data = {
    'event': 'subscribe',
    'pair': pair
}

sio.emit(FETCH_CANDLESTICK_CS_PRO, subscribe_data, namespace=namespace)
```

```
RESPONSE
Sample response
{
  "o": "19.000000000000", //open
  "h": "19.000000000000", //high
  "l": "19.000000000000", //low
  "c": "19.000000000000", // close
  "volume": "470.300000000000",
  "end_time": "1699423980000",
  "symbol": "BAT,INR",
  "interval": "1"
}
```

```
import socketio
sio = socketio.Client()

pair = "BTC,INR"
sio.connect(url=base_url, namespaces=[namespace], transports='websocket',
            socketio_path='/pro/realtime-rates-socket/spot', wait=True, wait_timeout=10)
```

WebSockets (User data)

These websockets can be used to get order updates and balance updates for user orders that are placed through API trading. These socket connections needs to be authenticated by API key and secret.

Order update

This event provides the order details every time the status of the order changes.

Event Name = "FETCH_ORDER_UPDATES"

How to connect with order updates ?

Socket Config	Value
Type	Socket.IO
Version	v4
Handshake Path	/pro/realtime-rates-socket/spot/order-updates
Base URL	wss://ws.coinswitch.co/
namespace	orderupdates

Parameter	Type	Mandatory	Description
event	string	Yes	(case sensitive) Allowed values "subscribe"
apikey	string	Yes	(case sensitive) This is the api key that is send in other api calls for authentication

Balance update

This Event gives the coin/wallet balance information whenever there is an update to an existing order.

Event Name = "FETCH_BALANCE_UPDATES"

How to connect with balance updates ?

Socket Config	Value
Type	Socket.IO
Version	v4
Handshake Path	/pro/realtime-rates-socket/spot/balance-updates
Base URL	wss://ws.coinswitch.co/
namespace	balanceupdates

python java

```
'pair': pair
}
sio.emit(FETCH_TRADES_CS_PRO, subscribe_data, namespace=namespace)

RESPONSE
{
  "E": 1673257880000, // Event time
  "p": "1472499.0",   // Price
  "q": "0.0004",     // Quantity
  "s": "BTC,INR",    // Symbol
  "t": 395531528,    // Trade ID
  "e": "wazirx"      // Exchange
}
```

import socketio

BASE_URL = "https://ws.coinswitch.co"
HANDSHAKE_PATH = "/pro/realtime-rates-socket/spot/order-updates"
NAMESPACE = "/orderupdates"

sio = socketio.Client()

@sio.event(namespace=NAMESPACE)
def connect():
 print("Connected to namespace", NAMESPACE)

Send subscription event
sio.emit(
 "FETCH_ORDER_UPDATES",
 {
 "apikey": "<your api key>",
 "event": "subscribe"
 },
 namespace=NAMESPACE
)

@sio.on("FETCH_ORDER_UPDATES", namespace=NAMESPACE)
def on_order_update(data):
 print("Order update:", data)

@sio.event(namespace=NAMESPACE)
def disconnect():
 print("Disconnected from", NAMESPACE)

if __name__ == "__main__":
 sio.connect(
 BASE_URL,
 namespaces=[NAMESPACE],
 socketio_path=HANDSHAKE_PATH.lstrip("/")
)
 sio.wait()

RESPONSE
Sample response
{
 'E': 1716390819344, // Event time
 'O': 1716390819339, // Updated time
 'e': 'COINSWITCHX', // Exchange name
 'S': 'SELL', // side
 'V': '0.0003', //expected Base quantity
 'X': 'OPEN', //order status
 'i': '35286cf0-8c63-499f-bb43-86b326e4aa38', // order-id
 'o': 'LIMIT', // order type
 'p': '6036000', // expected_price
 's': 'btcinr', // symbol
 'v': '0', // actual_price
 'z': '0' // actual_base_quantity
}

TAKE_PROFIT_MARKET and STOP_MARKET orders
(Example :- true/false)

Note :- Use Take_Profit_Market and Stop_Market Orders for setting up Position TP/SL(Applied to full position). Send quantity as 0 in case of Take_Profit_Market and Stop_Market Orders

In response Order if Order Status is Raised, Query GET orders API with the order_id to get actual status.

In case if need to partially close any position, it is advised use reduce_only as true so that new position is not opened.

Remember — Use Valid API key and Signature!

Cancel Order

Use the following endpoint to cancel an order on the exchange:

HTTP Request

METHOD DELETE

ENDPOINT https://coinswitch.co/trade/api/v2/futures/order

REQUEST LIMIT: 10 requests per 60 seconds

Parameter	Type	Mandatory	Description
order_id	string	Yes	order_id that needs to be cancelled
exchange	string	Yes	Allowed values: "EXCHANGE_2" (case insensitive)

Remember — Use valid API key and signature!

Update Leverage

Use the following endpoint to update leverage of a coin on the exchange:

HTTP Request

METHOD POST

ENDPOINT https://coinswitch.co/trade/api/v2/futures/leverage

REQUEST LIMIT: 10 requests per 60 seconds

Parameter	Type	Mandatory	Description
symbol	string	Yes	Asset symbol whose leverage needs to be updated (BTCUSDT, ETHUSDT etc.)
exchange	string	Yes	Allowed values: "EXCHANGE_2" (case insensitive)
leverage	integer	Yes	Value of leverage that is to be set for the given symbol (should be an integer between 1 to max leverage)

Note :- In case of Open Position or Open Orders , Leverage change is not allowed.

Get Max allowed leverage for an asset from instrument_info API

Remember — Use valid API key and signature!

GET Leverage

Use the following endpoint to get leverage of a coin on the exchange:

HTTP Request

METHOD GET

ENDPOINT https://coinswitch.co/trade/api/v2/futures/leverage

REQUEST LIMIT: 20 requests per 60 seconds

Parameter	Type	Mandatory	Description
symbol	string	Yes	Asset symbol whose leverage needs to be updated (BTCUSDT, ETHUSDT etc.)
exchange	string	Yes	Allowed values: "EXCHANGE_2" (case insensitive)

Remember — Use valid API key and signature!

python

java

```
{
  "message": "Invalid access"
}
```

```
import requests
import json

url = "https://coinswitch.co/trade/api/v2/futures/order"

payload = {
  "exchange": "EXCHANGE_2"
  "order_id": "698ed406-8ef5-4664-9779-f7978702a447"
}

headers = {
  'Content-Type': 'application/json',
  'X-AUTH-SIGNATURE': <api-signature>, #https://api-trading.coinswitch.co/#
  'X-AUTH-APIKEY': <api-key>
}
try:
  response = requests.request("DELETE", url, headers=headers, json=payload)
  print("Response JSON:", response.json())
except Exception as e:
  print(f"An error occurred: {e}")
```

The above command returns JSON structured like this:

```
STATUS CODE 200
{
  "data": {
    "order_id": "0193688e-5212-7493-be58-4f83644772e8",
    "exchange": "EXCHANGE_2",
    "symbol": "DOGEUSDT",
    "side": "BUY",
    "order_type": "LIMIT",
    "status": "CANCELLATION_RAISED",
    "quantity": "22",
    "exec_quantity": "0",
    "price": "0.28",
    "avg_exec_price": "0",
    "exec_fee": "0",
    "reduce_only": false,
    "created_at": 1732625977884,
    "updated_at": 1732626020104
  }
}
```

```
import requests
import json

url = "https://coinswitch.co/trade/api/v2/futures/leverage"

payload = {
  "symbol" : "btcusdt",
  "exchange" : "EXCHANGE_2",
  "leverage" : 13,
}

headers = {
  'Content-Type': 'application/json',
  'X-AUTH-SIGNATURE': <api-signature>, #https://api-trading.coinswitch.co/#
  'X-AUTH-APIKEY': <api-key>
}

try:
  response = requests.request("POST", url, headers=headers, json=payload)
  print("Response JSON:", response.json())
except Exception as e:
  print(f"An error occurred: {e}")
```

The above command returns JSON structured like this:

```
STATUS CODE 200
{
  "data": {
    "exchange": "EXCHANGE_2",
    "symbol": "BTCUSDT",
    "leverage": "13"
  }
}
```

```
import requests
import json

params = {
  "symbol": "btcusdt",
  "exchange": "EXCHANGE_2"
}

payload = {}

endpoint = "/trade/api/v2/futures/leverage"

endpoint += ('&', '?')[urlparse(endpoint).query == ''] + urlencode(params)

url = "https://coinswitch.co" + endpoint

headers = {
  'Content-Type': 'application/json',
  'X-AUTH-SIGNATURE': <api-signature>, #https://api-trading.coinswitch.co/#
  'X-AUTH-APIKEY': <api-key>
}
try:
  response = requests.request("GET", url, headers=headers, json=params)
  print("Response JSON:", response.json())
except Exception as e:
  print(f"An error occurred: {e}")
```

Use the following endpoint to get open positions on the exchange:

HTTP Request

METHOD GET

ENDPOINT `https://coinswitch.co/trade/api/v2/futures/positions`

REQUEST LIMIT: 20 requests per 60 seconds

Request Parameters

Parameter	Type	Mandatory	Description
exchange	string	Yes	Allowed values: "EXCHANGE_2" (case insensitive)
symbol	string	Yes	Asset symbol whose leverage needs to be updated (BTCUSDT, ETHUSDT etc.)

Note :- If mark price/ltp price is zero and unrealised pnl is zero, please do not consider this value. It means there is an issue with the system. Please expect other fields to be correct.

Remember — Use valid API key and signature!

Use the following endpoint to get transactions on the exchange:

HTTP Request

METHOD GET

ENDPOINT `https://coinswitch.co/trade/api/v2/futures/transactions`

REQUEST LIMIT: 20 requests per 60 seconds

Request Parameters

Parameter	Type	Mandatory	Description
exchange	string	Yes	Allowed values: "EXCHANGE_2" (case insensitive)
symbol	string	No	Asset symbol whose leverage needs to be updated (BTCUSDT, ETHUSDT etc.)
type	string	No	Allowed values: "commission", P&L, funding fee, liquidation fee"
transaction_id	string	No	Transaction ID of the transaction you want to see

Remember — Use valid API key and signature!

pythonjava

STATUS CODE 200

```
{
  "data": {
    "exchange": "EXCHANGE_2",
    "symbol": "BTCUSDT",
    "leverage": "13"
  }
}
```

```
import requests
import json

params = {
    "exchange": "EXCHANGE_2",
    "symbol": "btcusdt"
}

payload = {}

endpoint = "/trade/api/v2/futures/positions"

endpoint += ('&', '?')[urlparse(endpoint).query == ''] + urlencode(params)

url = "https://coinswitch.co" + endpoint

headers = {
    'Content-Type': 'application/json',
    'X-AUTH-SIGNATURE': <api-signature>, #https://api-trading.coinswitch.co/#
    'X-AUTH-APIKEY': <api-key>
}

try :
    response = requests.request("GET", url, headers=headers, json=params)
    print("Response JSON:", response.json())
except Exception as e:
    print(f"An error occurred: {e}")
```

The above command returns JSON structured like this:

STATUS CODE 200

```
{
  "data": [
    {
      "exchange": "EXCHANGE_2",
      "position_id": "8b81b763-df36-4c93-9bc8-9a93d65b8546",
      "symbol": "DOGEUSDT",
      "position_side": "LONG",
      "leverage": "25",
      "position_size": "65", // in base_quantity (ex. 65 DOGE)
      "position_value": "25.09455", // in USDT
      "position_margin": "34.052117186199", // in USDT
      "maint_margin": "37.641825", // minimum margin balance after ad
      "avg_entry_price": "0.38607",
      "mark_price": "0.38231",
      "last_price": "0.38135",
      "unrealised_pnl": "0.3068",
      "liquidation_price": "0.129122150942",
      "status": "OPEN",
      "created_at": 1732617093684,
      "updated_at": 1732636761825
    },
    {
      "exchange": "EXCHANGE_2",
      "position_id": "678708aa-507b-4881-8b3f-1a50b63dd0c4",
      "symbol": "ADAUSDT",
      "position_side": "SHORT",
      "leverage": "25",
      "position_size": "11",
      "position_value": "10.9681",
      "position_margin": "6.779670888571",
      "maint_margin": "15.6783",
      "avg_entry_price": "0.9971",
      "mark_price": "0.9502",
      "last_price": "0.9503",
      "unrealised_pnl": "-0.5148",
      "liquidation_price": "1.577930285714",
      "status": "OPEN",
      "created_at": 1732613252041,
      "updated_at": 1732636762998
    }
  ]
}
```

```
import requests
import json

params = {
    "symbol": "btcusdt",
    "exchange": "EXCHANGE_2",
    "type": "commission",
    "transaction_id" : "c47d28b9-0e68-56e8-892c-5ff1ce5eef3e"
}

payload = {}

endpoint = "/trade/api/v2/futures/transactions"

endpoint += ('&', '?')[urlparse(endpoint).query == ''] + urlencode(params)

url = "https://coinswitch.co" + endpoint

headers = {
    'Content-Type': 'application/json',
    'X-AUTH-SIGNATURE': <api-signature>, #https://api-trading.coinswitch.co/#
    'X-AUTH-APIKEY': <api-key>
}

try:
    response = requests.request("GET", url, headers=headers, json=payload)
    print("Response JSON:", response.json())
except Exception as e:
    print(f"An error occurred: {e}")
```

The above command returns JSON structured like this:

STATUS CODE 200

```
{
  "data": [
    {
      "exchange": "EXCHANGE_2",
      "transaction_id": "678708aa-507b-4881-8b3f-1a50b63dd0c4-01936770",
      "symbol": "ADAUSDT",
      "type": "FUNDING_FEE",
      "quote_asset": "USDT",
      "amount": "0.00099946"
    },
    {
      "exchange": "EXCHANGE_2",
      "transaction_id": "8b81b763-df36-4c93-9bc8-9a93d65b8546-01936770",
      "symbol": "DOGEUSDT",
      "type": "FUNDING_FEE",
      "quote_asset": "USDT",
      "amount": "-0.002482813801"
    }
  ],
  {
```


ADD Margin

Use the following endpoint to add margin to an order on the exchange:

HTTP Request

METHOD

POST

ENDPOINT

https://coinswitch.co/trade/api/v2/futures/add_margin

REQUEST LIMIT:

10 requests per 60 seconds

Request Parameters

Parameter	Type	Mandatory	Description
symbol	string	Yes	Asset symbol whose leverage needs to be updated (BTCUSDT, ETHUSDT etc.)
exchange	string	Yes	Allowed values: "EXCHANGE_2" (case insensitive)
margin	integer	Yes	Value of margin that is to be set for the given symbol

Note :- Margin amount should be less than or equal to available balance.

✔ Remember — Use valid API key and signature!

Cancel All Open Orders

Use the following endpoint to cancel all open orders on the exchange:

HTTP Request

METHOD

POST

ENDPOINT

https://coinswitch.co/trade/api/v2/futures/cancel_all

REQUEST LIMIT:

10 requests per 60 seconds

Request Parameters

Parameter	Type	Mandatory	Description
exchange	string	Yes	Allowed values: "EXCHANGE_2" (case insensitive)
symbol	string	No	Asset symbol whose orders needs to be cancelled (BTCUSDT, ETHUSDT etc.)

Note :- Will cancel all open orders including Take_Profit_Market/Stop_Market(TP/SL)

Query the specific order_id received in response via get order (Status) API to get the exact status of the order.

✔ Remember — Use valid API key and signature!

GET Wallet Balance

Use the following endpoint to get your wallet balance:

HTTP Request

METHOD

GET

ENDPOINT

https://coinswitch.co/trade/api/v2/futures/wallet_balance

REQUEST LIMIT:

20 requests per 60 seconds

Request Parameters

There are no parameters required for this API.

✔ Remember — Use valid API key and signature!

pythonjava

```
"symbol": "DOGEUSDT",
"type": "ADD_MARGIN",
"quote_asset": "USDT",
"amount": "16"
},
{
  "exchange": "EXCHANGE_2",
  "transaction_id": "8b81b763-df36-4c93-9bc8-9a93d65b8546",
  "symbol": "DOGEUSDT",
  "type": "ADD_MARGIN",
  "quote_asset": "USDT",
  "amount": "16"
}
]
```

```
import requests
import json

url = "https://coinswitch.co/trade/api/v2/futures/add_margin"

payload = {
  "exchange" : "EXCHANGE_2",
  "symbol" : "dogeusdt",
  "margin" : 16
}

headers = {
  'Content-Type': 'application/json',
  'X-AUTH-SIGNATURE': <api-signature>, #https://api-trading.coinswitch.co/#
  'X-AUTH-APIKEY': <api-key>
}

try:
  response = requests.request("POST", url, headers=headers, json=payload)
  print("Response JSON:", response.json())
except Exception as e:
  print(f"An error occurred: {e}")
```

The above command returns JSON structured like this:

```
STATUS CODE 200
{
  "data": {
    "exchange": "EXCHANGE_2",
    "position_id": "8b81b763-df36-4c93-9bc8-9a93d65b8546",
    "symbol": "DOGEUSDT",
    "position_side": "LONG",
    "leverage": "25",
    "position_size": "65",
    "position_value": "25.09455",
    "position_margin": "50.052117186199",
    "maint_margin": "37.641825",
    "avg_entry_price": "0.38607",
    "mark_price": "0.38694",
    "last_price": "0.38677",
    "unrealised_pnl": "-0.0455",
    "liquidation_price": "-0.3752759970953692",
    "status": "OPEN",
    "created_at": 1732617093684,
    "updated_at": 1732636761825
  }
}
```

```
import requests
import json

url = "https://coinswitch.co/trade/api/v2/futures/cancel_all"

payload = {
  "exchange" : "EXCHANGE_2",
}

headers = {
  'Content-Type': 'application/json',
  'X-AUTH-SIGNATURE': <api-signature>, #https://api-trading.coinswitch.co/#
  'X-AUTH-APIKEY': <api-key>
}

try:
  response = requests.request("POST", url, headers=headers, json=payload)
  print("Response JSON:", response.json())
except Exception as e:
  print(f"An error occurred: {e}")
```

The above command returns JSON structured like this:

```
STATUS CODE 200
{
  "data": {
    "orders_ids": [
      "01936be4-2571-7a3f-97fa-a29a2b85d717",
      "01936be4-9e37-7487-8e31-20fede6ef271"
    ]
  }
}
```

```
import requests
import json

url = "https://coinswitch.co/trade/api/v2/futures/wallet_balance"

payload = {}

headers = {
  'Content-Type': 'application/json',
  'X-AUTH-SIGNATURE': <api-signature>, #https://api-trading.coinswitch.co/#
  'X-AUTH-APIKEY': <api-key>
}

try:
  response = requests.request("GET", url, headers=headers, json=payload)
  print("Response JSON:", response.json())
except Exception as e:
  print(f"An error occurred: {e}")
```

The above command returns JSON structured like this:

```
STATUS CODE 200
{
  "data": {
    "base_asset_balances": [
      {
        "base_asset": "USDT",
        "balances": {
          "total_balance": "60960.82599588",
          "total_available_balance": "60910.77418715",
```

Get Order (Status)

Use the following endpoint to get status of an order on the exchange:

HTTP Request

METHOD GET

ENDPOINT `https://coinswitch.co/trade/api/v2/futures/order`

REQUEST LIMIT: 20 requests per 60 seconds

Request Parameters

Parameter	Type	Mandatory	Description
order_id	string	Yes	order_id whose information is required.

Remember — Use valid API key and signature!

Open Order

Use the following endpoint to get open orders on the exchange:

HTTP Request

METHOD POST

ENDPOINT `https://coinswitch.co/trade/api/v2/futures/orders/open`

python **java**

```
"total_open_order_margin": "-6.778517"
    }
  }
],
"asset": [
  {
    "symbol": "DASHUSD",
    "base_asset": "USD",
    "exchange": "EXCHANGE_2",
    "blocked_balance": "0",
    "position_margin": "0",
    "open_order_margin": "0"
  },
  {
    "symbol": "1000PEPEUSD",
    "base_asset": "USD",
    "exchange": "EXCHANGE_2",
    "blocked_balance": "0",
    "position_margin": "0",
    "open_order_margin": "0"
  },
  {
    "symbol": "BTCUSD",
    "base_asset": "USD",
    "exchange": "EXCHANGE_2",
    "blocked_balance": "0",
    "position_margin": "0",
    "open_order_margin": "0"
  },
  {
    "symbol": "ETHUSD",
    "base_asset": "USD",
    "exchange": "EXCHANGE_2",
    "blocked_balance": "0",
    "position_margin": "0",
    "open_order_margin": "0"
  },
  {
    "symbol": "1000FLOKUSD",
    "base_asset": "USD",
    "exchange": "EXCHANGE_2",
    "blocked_balance": "0",
    "position_margin": "0",
    "open_order_margin": "0"
  },
  {
    "symbol": "HMSTRUSD",
    "base_asset": "USD",
    "exchange": "EXCHANGE_2",
    "blocked_balance": "0.00006589",
    "position_margin": "0",
    "open_order_margin": "0.00006589"
  },
  {
    "symbol": "DOGEUSD",
    "base_asset": "USD",
    "exchange": "EXCHANGE_2",
    "blocked_balance": "50.04968529",
    "position_margin": "50.04959675",
    "open_order_margin": "0.00008854"
  },
  {
    "symbol": "EIGENUSD",
    "base_asset": "USD",
    "exchange": "EXCHANGE_2",
    "blocked_balance": "0",
    "position_margin": "0",
    "open_order_margin": "0"
  },
  {
    "symbol": "ADAUSD",
    "base_asset": "USD",
    "exchange": "EXCHANGE_2",
    "blocked_balance": "0.00205755",
    "position_margin": "6.78072898",
    "open_order_margin": "-6.77867143"
  }
]
}
}
```

```
import requests
import json

params = {
    "order_id": "698ed406-8ef5-4664-9779-f7978702a447",
}
payload = {}

endpoint = "/trade/api/v2/futures/order"

endpoint += ('&', '?')[urlparse(endpoint).query == ''] + urlencode(params)

url = "https://coinswitch.co" + endpoint

headers = {
    'Content-Type': 'application/json',
    'X-AUTH-SIGNATURE': '<api-signature>', #https://api-trading.coinswitch.co/#
    'X-AUTH-APIKEY': '<api-key>'
}

try:
    response = requests.request("GET", url, headers=headers, json=payload)
    print("Response JSON:", response.json())
except Exception as e:
    print(f"An error occurred: {e}")
```

The above command returns JSON structured like this:

```
STATUS CODE 200
{
  "data": {
    "order": {
      "order_id": "698ed406-8ef5-4664-9779-f7978702a447",
      "exchange": "EXCHANGE_2",
      "symbol": "DOGEUSD",
      "side": "BUY",
      "status": "CANCELLED",
      "order_type": "LIMIT",
      "quantity": "6.16",
      "exec_quantity": "0",
      "price": "0.28",
      "avg_execution_price": "0",
      "execution_fee": "0.0041",
      "realised_pnl": "0",
      "reduce_only": false,
      "created_at": 1732623664116,
      "updated_at": 1732623664116
    }
  }
}
```

```
import requests
import json

payload = {
    "exchange": "EXCHANGE_2",
    "symbol" : "btcusdt"
}
```

REQUEST LIMIT: 20 requests per 60 seconds

Request Parameters

Parameter	Type	Mandatory	Description
exchange	string	Yes	Allowed values: "EXCHANGE_2" (case insensitive)
symbol	string	No	Asset symbol whose open orders needs to be seen (BTCUSDT, ETHUSDT etc.)
limit	integer	No	Limit for data size (Number of orders to fetch)
from_time	integer	No	To get open orders placed after a specific time (Time in milliseconds)
to_time	integer	No	To get open orders placed before a specific time (Time in milliseconds)

Note :- Please provide both from_time and to_time to get open orders in between a specific set of times.

The maximum number for limit is 50.

The difference between from_time and to_time cannot be more than 7 days. (If from_time or to_time is not given, the orders shown will be the last 50 orders from the last 7 days)

✔ Remember — Use valid API key and signature!

Closed Order

Use the following endpoint to get closed orders on the exchange:

HTTP Request

METHOD POST

ENDPOINT https://coinswitch.co/trade/api/v2/futures/orders/closed

REQUEST LIMIT: 20 requests per 60 seconds

Request Parameters

Parameter	Type	Mandatory	Description
exchange	string	Yes	Allowed values: "EXCHANGE_2" (case insensitive)
symbol	string	No	Asset symbol whose closed orders needs to be seen (BTCUSDT, ETHUSDT etc.)
limit	integer	No	Limit for data size (Number of orders to fetch)
from_time	integer	No	To get closed orders placed after a specific time (Time in milliseconds)
to_time	integer	No	To get closed orders placed before a specific time (Time in milliseconds)

Note :- Please provide both from_time and to_time to get closed orders in between a specific set of times.

The maximum number for limit is 50.

The difference between from_time and to_time cannot be more than 7 days. (If from_time or to_time is not given, the orders shown will be the last 50 orders from the last 7 days)

✔ Remember — Use valid API key and signature!

Get Instrument Info

Use the following endpoint to get instrument info on the exchange:

HTTP Request

METHOD GET

ENDPOINT https://coinswitch.co/trade/api/v2/futures/instrument_info

REQUEST LIMIT: 100 requests per 60 seconds

Request Parameters

Parameter	Type	Mandatory	Description
exchange	string	No	Allowed values: "EXCHANGE_2" (case insensitive)

✔ Remember — Use valid API key and signature!

python java

```
headers = {
    'Content-Type': 'application/json',
    'X-AUTH-SIGNATURE': <api-signature>, #https://api-trading.coinswitch.co/#
    'X-AUTH-APIKEY': <api-key>
}

try:
    response = requests.request("POST", url, headers=headers, json=payload)
    print("Response JSON:", response.json())
except Exception as e:
    print(f"An error occurred: {e}")
```

The above command returns JSON structured like this:

STATUS CODE 200

{
 "data": {
 "orders": [
 {
 "order_id": "01936c09-85a6-79d5-a7b8-d0e19daa9fbf",
 "exchange": "EXCHANGE_2",
 "symbol": "BTCUSDT",
 "side": "BUY",
 "status": "RAISED",
 "order_type": "LIMIT",
 "quantity": "160",
 "exec_quantity": "0",
 "price": "80000",
 "avg_execution_price": "0",
 "execution_fee": "0.1051",
 "realised_pnl": "0",
 "reduce_only": false,
 "created_at": 1732684383674,
 "updated_at": 1732684383674
 }
],
 "cursor": 1732684383674
 }
}

import requests
import json

payload = {
 "exchange": "EXCHANGE_2",
 "symbol": "btcusdt",
 "limit": 10
}

url = "https://coinswitch.co/trade/api/v2/futures/orders/closed"

headers = {
 'Content-Type': 'application/json',
 'X-AUTH-SIGNATURE': <api-signature>, #https://api-trading.coinswitch.co/#
 'X-AUTH-APIKEY': <api-key>
}

try:
 response = requests.request("POST", url, headers=headers, json=payload)
 print("Response JSON:", response.json())
except Exception as e:
 print(f"An error occurred: {e}")

The above command returns JSON structured like this:

STATUS CODE 200

{
 "data": {
 "orders": [
 {
 "order_id": "01936be4-9e37-7487-8e31-20fede6ef271",
 "exchange": "EXCHANGE_2",
 "symbol": "BTCUSDT",
 "side": "BUY",
 "status": "CANCELLED",
 "order_type": "LIMIT",
 "quantity": "160",
 "exec_quantity": "0",
 "price": "80000",
 "avg_execution_price": "0",
 "execution_fee": "0.1051",
 "realised_pnl": "0",
 "reduce_only": false,
 "created_at": 1732681965292,
 "updated_at": 1732681965292
 },
 {
 "order_id": "01936be4-2571-7a3f-97fa-a29a2b85d717",
 "exchange": "EXCHANGE_2",
 "symbol": "DOGEUSDT",
 "side": "BUY",
 "status": "CANCELLED",
 "order_type": "LIMIT",
 "quantity": "6.16",
 "exec_quantity": "0",
 "price": "0.28",
 "avg_execution_price": "0",
 "execution_fee": "0.0041",
 "realised_pnl": "0",
 "reduce_only": false,
 "created_at": 1732681934263,
 "updated_at": 1732681934263
 }
],
 "cursor": 1732616972834
 }
}

import requests
import json

params = {
 "exchange": "EXCHANGE_2",
}

payload = {}

endpoint = "/trade/api/v2/futures/instrument_info"

endpoint += ('&', '?')[urlparse(endpoint).query == ''] + urlencode(params)

url = "https://coinswitch.co" + endpoint

headers = {
 'Content-Type': 'application/json',
 'X-AUTH-SIGNATURE': <api-signature>, #https://api-trading.coinswitch.co/#
 'X-AUTH-APIKEY': <api-key>
}

try:
 response = requests.request("GET", url, headers=headers, json=payload)
 print("Response JSON:", response.json())
except Exception as e:
 print(f"An error occurred: {e}")

coinswitchPRO
Q Search
Change Log
Streamline Your Trading Expe...
Setting up an API Key
Information about our APIs
How to Setup
Get Set Go
Signature Generation
Keys Validation
Ping Testing
Introduction to C2C Trading
SPOT Trading APIs
Error Scenarios
WebSockets (Market Data)
Client Set Up for Websockets
WebSockets (User data)
Futures Trading APIs
Futures Public APIs
Futures WebSockets
Options Trading APIs
HFT (High-Frequency Trading)
HFT APIs
HFT Sockets
Terms and Conditions (T&Cs)
Contact us
Sign Up for a Developer Key
Documentation Powered by Slate

python

java

STATUS CODE 200

```
{
  "data": {
    "1000FLOKIUSD": {
      "symbol": "1000floki",
      "base_asset": "1000floki",
      "quote_asset": "usdt",
      "status": "TRADING",
      "type": "PERPETUAL_FUTURES",
      "min_leverage": "1",
      "max_leverage": "25",
      "leverage_step": 1,
      "min_base_quantity": "35",
      "base_quantity_step_size": "1",
      "lot_size": "1",
      "quantity_precision": 0,
      "price_precision": 5,
      "tick_size": 5,
      "max_market_base_quantity": "1443800",
      "max_base_quantity": "7245440",
      "risk_limit": "312120.69",
      "maint_margin_rate": "2",
      "taker_fee_rate": "0.00065",
      "maker_fee_rate": "0.00024",
      "liq_fee_rate": "0.0075",
      "quote_asset_precision": 5
    },
    "1000PEPEUSD": {
      "symbol": "1000pepe",
      "base_asset": "1000pepe",
      "quote_asset": "usdt",
      "status": "TRADING",
      "type": "PERPETUAL_FUTURES",
      "min_leverage": "1",
      "max_leverage": "25",
      "leverage_step": 1,
      "min_base_quantity": "500",
      "base_quantity_step_size": "100",
      "lot_size": "100",
      "quantity_precision": -2,
      "price_precision": 7,
      "tick_size": 7,
      "max_market_base_quantity": "112500000",
      "max_base_quantity": "106977041",
      "risk_limit": "400000",
      "maint_margin_rate": "2",
      "taker_fee_rate": "0.00065",
      "maker_fee_rate": "0.00024",
      "liq_fee_rate": "0.0075",
      "quote_asset_precision": 7
    },
    "ACHUSD": {
      "symbol": "ach",
      "base_asset": "ach",
      "quote_asset": "usdt",
      "status": "TRADING",
      "type": "PERPETUAL_FUTURES",
      "min_leverage": "1",
      "max_leverage": "20",
      "leverage_step": 1,
      "min_base_quantity": "410",
      "base_quantity_step_size": "10",
      "lot_size": "10",
      "quantity_precision": -1,
      "price_precision": 6,
      "tick_size": 6,
      "max_market_base_quantity": "1884000",
      "max_base_quantity": "7907200",
      "risk_limit": "34886.03",
      "maint_margin_rate": "3",
      "taker_fee_rate": "0.00065",
      "maker_fee_rate": "0.00024",
      "liq_fee_rate": "0.0075",
      "quote_asset_precision": 6
    },
    "ADAUSD": {
      "symbol": "ada",
      "base_asset": "ada",
      "quote_asset": "usdt",
      "status": "TRADING",
      "type": "PERPETUAL_FUTURES",
      "min_leverage": "1",
      "max_leverage": "25",
      "leverage_step": 1,
      "min_base_quantity": "8",
      "base_quantity_step_size": "1",
      "lot_size": "1",
      "quantity_precision": 0,
      "price_precision": 4,
      "tick_size": 4,
      "max_market_base_quantity": "1109600",
      "max_base_quantity": "5052400",
      "risk_limit": "300000",
      "maint_margin_rate": "1.5",
      "taker_fee_rate": "0.00065",
      "maker_fee_rate": "0.00024",
      "liq_fee_rate": "0.0075",
      "quote_asset_precision": 4
    },
    "BTCUSD": {
      "symbol": "btc",
      "base_asset": "btc",
      "quote_asset": "usdt",
      "status": "TRADING",
      "type": "PERPETUAL_FUTURES",
      "min_leverage": "1",
      "max_leverage": "25",
      "leverage_step": 1,
      "min_base_quantity": "0.001",
      "base_quantity_step_size": "0.001",
      "lot_size": "0.001",
      "quantity_precision": 3,
      "price_precision": 2,
      "tick_size": 1,
      "max_market_base_quantity": "119",
      "max_base_quantity": "952",
      "risk_limit": "1000000",
      "maint_margin_rate": "0.56",
      "taker_fee_rate": "0.00065",
      "maker_fee_rate": "0.00024",
      "liq_fee_rate": "0.0075",
      "quote_asset_precision": 1
    },
    "DASHUSD": {
      "symbol": "dash",
      "base_asset": "dash",
      "quote_asset": "usdt",
      "status": "TRADING",
      "type": "PERPETUAL_FUTURES",
      "min_leverage": "1",
      "max_leverage": "20",
      "leverage_step": 1,
      "min_base_quantity": "0.22",
      "base_quantity_step_size": "0.01",
      "lot_size": "0.01",
      "quantity_precision": 2,
      "price_precision": 2,
      "tick_size": 2,
      "max_market_base_quantity": "1672",
      "max_base_quantity": "6904",
      "risk_limit": "58687.2",
      "maint_margin_rate": "2.5",
      "taker_fee_rate": "0.00065",
      "maker_fee_rate": "0.00024",
      "liq_fee_rate": "0.0075",
      "quote_asset_precision": 2
    },
    "DOGEUSD": {
      "symbol": "doge",
      "base_asset": "doge",
      "quote_asset": "usdt",
```

Sign Up for a Developer Key

Documentation Powered by Slate

Futures Public APIs

GET Order Book

Use the following endpoint to get order book details of an asset on the exchange:

HTTP Request

METHOD `GET`

ENDPOINT `https://coinswitch.co/trade/api/v2/futures/order_book`

REQUEST LIMIT: `100 requests per 60 seconds`

Request Parameters

Parameter	Type	Mandatory	Description
exchange	string	Yes	Allowed values: "EXCHANGE_2" (case insensitive)
symbol	string	Yes	Asset symbol whose closed orders needs to be seen (BTCUSD, ETHUSD etc.)

Remember — Use valid API key and signature!

pythonjava

```
"min_leverage": "1",
"max_leverage": "25",
"leverage_step": 1,
"min_base_quantity": "20",
"base_quantity_step_size": "1",
"lot_size": "1",
"quantity_precision": 0,
"price_precision": 5,
"tick_size": 5,
"max_market_base_quantity": "5991100",
"max_base_quantity": "34943120",
"risk_limit": "500000",
"maint_margin_rate": "1.5",
"taker_fee_rate": "0.00065",
"maker_fee_rate": "0.00024",
"liq_fee_rate": "0.0075",
"quote_asset_precision": 5
},
"EIGENUSD": {
  "symbol": "eigen",
  "base_asset": "eigen",
  "quote_asset": "usdt",
  "status": "TRADING",
  "type": "PERPETUAL_FUTURES",
  "min_leverage": "1",
  "max_leverage": "25",
  "leverage_step": 1,
  "min_base_quantity": "3",
  "base_quantity_step_size": "1",
  "lot_size": "1",
  "quantity_precision": 0,
  "price_precision": 4,
  "tick_size": 4,
  "max_market_base_quantity": "5400",
  "max_base_quantity": "21600",
  "risk_limit": "18472.32",
  "maint_margin_rate": "1.11",
  "taker_fee_rate": "0.00065",
  "maker_fee_rate": "0.00024",
  "liq_fee_rate": "0.0075",
  "quote_asset_precision": 4
},
"ETHUSD": {
  "symbol": "eth",
  "base_asset": "eth",
  "quote_asset": "usdt",
  "status": "TRADING",
  "type": "PERPETUAL_FUTURES",
  "min_leverage": "1",
  "max_leverage": "25",
  "leverage_step": 1,
  "min_base_quantity": "0.01",
  "base_quantity_step_size": "0.01",
  "lot_size": "0.01",
  "quantity_precision": 2,
  "price_precision": 2,
  "tick_size": 2,
  "max_market_base_quantity": "724",
  "max_base_quantity": "5792",
  "risk_limit": "1000000",
  "maint_margin_rate": "0.63",
  "taker_fee_rate": "0.00065",
  "maker_fee_rate": "0.00024",
  "liq_fee_rate": "0.0075",
  "quote_asset_precision": 2
},
"HMSTRUSD": {
  "symbol": "hmstr",
  "base_asset": "hmstr",
  "quote_asset": "usdt",
  "status": "TRADING",
  "type": "PERPETUAL_FUTURES",
  "min_leverage": "1",
  "max_leverage": "25",
  "leverage_step": 1,
  "min_base_quantity": "2030",
  "base_quantity_step_size": "10",
  "lot_size": "10",
  "quantity_precision": -1,
  "price_precision": 6,
  "tick_size": 6,
  "max_market_base_quantity": "1830000",
  "max_base_quantity": "7320000",
  "risk_limit": "6769.17",
  "maint_margin_rate": "1.11",
  "taker_fee_rate": "0.00065",
  "maker_fee_rate": "0.00024",
  "liq_fee_rate": "0.0075",
  "quote_asset_precision": 6
}
}
```

```
import requests
import json

params = {
    "symbol": "btcusdt",
    "exchange": "EXCHANGE_2"
}

payload = {}

endpoint = "/trade/api/v2/futures/order_book"

endpoint += ('&', '?')[urlparse(endpoint).query == ''] + urlencode(params)

url = "https://coinswitch.co" + endpoint

headers = {
    'Content-Type': 'application/json',
    'X-AUTH-SIGNATURE': <api-signature>, #https://api-trading.coinswitch.co/#
    'X-AUTH-APIKEY': <api-key>
}

try:
    response = requests.request("GET", url, headers=headers, json=params)
    print("Response JSON:", response.json())
except Exception as e:
    print(f"An error occurred: {e}")
```

The above command returns JSON structured like this:

```
STATUS CODE 200
{
  "data": {
    "timestamp": 1732685131699,
    "asks": [
      [
        "3388.92", //Price
        "0.10" //Quantity
      ],
      [
        "3388.94",
        "0.01"
      ],
      [
        "3389.02",
        "0.17"
      ]
    ],
    "bids": [
```

GET L2 Order Book

Use the following endpoint to get L2 order book details of an asset on the exchange:

HTTP Request

METHOD `GET`

ENDPOINT `https://coinswitch.co/trade/api/v2/futures/order_book`

REQUEST LIMIT: `100 requests per 60 seconds`

Note :- The Request limit is 100 for both order_book and L2 order_book combined.

Request Parameters

Parameter	Type	Mandatory	Description
exchange	string	Yes	Allowed values: "EXCHANGE_2" (case insensitive)
symbol	string	Yes	Asset symbol whose closed orders needs to be seen (BTCUSDT, ETHUSDT etc.)
l2Orderbook	string	Yes	Should be "true" to fetch l2Orderbook Data.

Remember — Use valid API key and signature!

GET Ticker

Use the following endpoint to get ticker data of an asset on the exchange:

HTTP Request

METHOD `GET`

ENDPOINT `https://coinswitch.co/trade/api/v2/futures/ticker`

REQUEST LIMIT: `100 requests per 60 seconds`

Request Parameters

Parameter	Type	Mandatory	Description
exchange	string	Yes	Allowed values: "EXCHANGE_2" (case insensitive)
symbol	string	Yes	Asset symbol whose leverage needs to be updated (BTCUSDT, ETHUSDT etc.)

Remember — Use valid API key and signature!

pythonjava

```
    "2.16" //Quantity
  ],
  [
    "3425.69",
    "0.01"
  ],
  [
    "3425.66",
    "0.01"
  ]
],
"symbol": "ETHUSDT"
}
}
```

```
import requests
import json

params = {
    "symbol": "btcusdt",
    "exchange": "EXCHANGE_2",
    "l2Orderbook": "true"
}

payload = {}

endpoint = "/trade/api/v2/futures/order_book"

endpoint += ('&', '?')[urlparse(endpoint).query == ''] + urlencode(params)

url = "https://coinswitch.co" + endpoint

headers = {
    'Content-Type': 'application/json',
    'X-AUTH-SIGNATURE': <api-signature>, #https://api-trading.coinswitch.co/#
    'X-AUTH-APIKEY': <api-key>
}

try:
    response = requests.request("GET", url, headers=headers, json=params)
    print("Response JSON:", response.json())
except Exception as e:
    print(f"An error occurred: {e}")
```

The above command returns JSON structured like this:

```
STATUS CODE 200
{
  "data": {
    "timestamp": 1734342843976,
    "asks": [
      [
        "3950.39",
        "14.9800"
      ],
      [
        "3950.40",
        "17.4900"
      ],
      [
        "3950.42",
        "20.7500"
      ],
      [
        "3950.45",
        "20.7600"
      ]
    ],
    "bids": [
      [
        "3950.41",
        "0.1500"
      ],
      [
        "3950.38",
        "0.1700"
      ],
      [
        "3950.34",
        "0.1800"
      ],
      [
        "3950.31",
        "0.1900"
      ]
    ],
    "symbol": "ETHUSDT"
  }
}
```

```
import requests
import json

params = {
    "symbol": "btcusdt",
    "exchange": "EXCHANGE_2"
}

payload = {}

endpoint = "/trade/api/v2/futures/ticker"

endpoint += ('&', '?')[urlparse(endpoint).query == ''] + urlencode(params)

url = "https://coinswitch.co" + endpoint

headers = {
    'Content-Type': 'application/json',
    'X-AUTH-SIGNATURE': <api-signature>, #https://api-trading.coinswitch.co/#
    'X-AUTH-APIKEY': <api-key>
}

try:
    response = requests.request("GET", url, headers=headers, json=params)
    print("Response JSON:", response.json())
except Exception as e:
    print(f"An error occurred: {e}")
```

The above command returns JSON structured like this:

```
STATUS CODE 200
{
  "data": {
    "EXCHANGE_2": {
      "low_price_24h": "94707.00",
      "high_price_24h": "97276.00",
      "last_price": "95136.60",
      "symbol": "BTCUSDT",
      "exchange": "EXCHANGE_2",
      "timestamp": 1732821806680,
      "best_ask_price": "95136.70",
      "best_bid_price": "95136.60",
      "price_24h_pcnt": "-1.297300",
      "base_asset_volume_24h": "93707.6800",
      "quote_asset_volume_24h": "8970676685.2608",
      "index_price": 95046.53,
      "mark_price": 95136.7,
```


KLines

Use the following endpoint to get klines on the exchange:

HTTP Request

METHOD GET

ENDPOINT `https://coinswitch.co/trade/api/v2/futures/klines`

REQUEST LIMIT: 30 requests per 60 seconds

Request Parameters

Parameter	Type	Mandatory	Description
exchange	string	Yes	Allowed values: "EXCHANGE_2" (case insensitive)
symbol	string	Yes	Asset symbol whose leverage needs to be updated (BTCUSDT, ETHUSDT etc.)
interval	string	Yes	Time interval in which data is given (These are the accepted interval values ['1', '5', '15', '30', '60', '120', '240', '360', '720', '1440'])
start_time	integer	No	Time after which data is required
end_time	integer	No	Time before which data is required

Remember — Use valid API key and signature!

Get Trades

Use the following endpoint to get trades data on the exchange:

HTTP Request

METHOD GET

ENDPOINT `https://coinswitch.co/trade/api/v2/futures/trades`

REQUEST LIMIT: 100 requests per 60 seconds

Request Parameters

Parameter	Type	Mandatory	Description
exchange	string	Yes	Allowed values: "EXCHANGE_2" (case insensitive)
symbol	string	Yes	Asset symbol whose leverage needs to be updated (BTCUSDT, ETHUSDT etc.)

Remember — Use valid API key and signature!

pythonjava

```
"funding_rate": 0.00039681,
"next_funding_timestamp": 1732838400000,
"best_bid_size": "3.189",
"best_ask_size": "3.963"
}
}
}
```

```
import requests
import json

params = {
    "symbol": "btcusdt",
    "exchange": "EXCHANGE_2",
    "interval": 1,
    "start_time": "1731555003792",
    "end_time": "1732555003792",
    "limit": 10
}

payload = {}

endpoint = "/trade/api/v2/futures/klines"

endpoint += ('&', '?')[urlparse(endpoint).query == ''] + urlencode(params)

url = "https://coinswitch.co" + endpoint

headers = {
    'Content-Type': 'application/json',
    'X-AUTH-SIGNATURE': <api-signature>, #https://api-trading.coinswitch.co/#
    'X-AUTH-APIKEY': <api-key>
}

try:
    response = requests.request("GET", url, headers=headers, json=params)
    print("Response JSON:", response.json())
except Exception as e:
    print(f"An error occurred: {e}")
```

The above command returns JSON structured like this:

```
STATUS CODE 200
{
  "data": [
    {
      "o": "95500.100000000000",
      "h": "95875.000000000000",
      "l": "94707.000000000000",
      "c": "95524.000000000000",
      "symbol": "BTCUSDT",
      "close_time": "1732795200000",
      "volume": "22426.480000000000",
      "start_time": "1732773600000",
      "interval": "360"
    },
    {
      "o": "95908.700000000000",
      "h": "96648.000000000000",
      "l": "94840.600000000000",
      "c": "95500.100000000000",
      "symbol": "BTCUSDT",
      "close_time": "1732773600000",
      "volume": "25083.398000000000",
      "start_time": "1732752000000",
      "interval": "360"
    }
  ]
}
```

```
import requests
import json

params = {
    "symbol": "btcusdt",
    "exchange": "EXCHANGE_2",
    "end_time": "1732555003792",
    "start_time": "1731555003792",
    "interval": "360"
}

payload = {}

endpoint = "/trade/api/v2/futures/trades"

endpoint += ('&', '?')[urlparse(endpoint).query == ''] + urlencode(params)

url = "https://coinswitch.co" + endpoint

headers = {
    'Content-Type': 'application/json',
    'X-AUTH-SIGNATURE': <api-signature>, #https://api-trading.coinswitch.co/#
    'X-AUTH-APIKEY': <api-key>
}

try:
    response = requests.request("GET", url, headers=headers, json=params)
    print("Response JSON:", response.json())
except Exception as e:
    print(f"An error occurred: {e}")
```

The above command returns JSON structured like this:

```
{
  "data": [
    {
      "E": 1732691693128, //timestamp
      "p": 0.39391, //price
      "q": 133, //quantity
      "e": "EXCHANGE_2", //exchange
      "s": "DOGEUSDT", //symbol
      "m": true //marketmaker
    },
    {
      "E": 1732691683370,
      "p": 0.39378,
      "q": 32,
      "e": "EXCHANGE_2",
      "s": "DOGEUSDT",
      "m": true
    },
    {
      "E": 1732691672521,
      "p": 0.3941,
      "q": 2286,
      "e": "EXCHANGE_2",
      "s": "DOGEUSDT",
      "m": true
    },
    {
      "E": 1732691662133,
      "p": 0.39401,
      "q": 1211,
      "e": "EXCHANGE_2",
      "s": "DOGEUSDT",
      "m": true
    }
  ]
}
```

Get All-Pairs Ticker

Use the following endpoint to get ticker data of all pairs on the exchange:

HTTP Request

METHOD GET

ENDPOINT `https://coinswitch.co/trade/api/v2/futures/all-pairs/ticker`

REQUEST LIMIT: 100 requests per 60 seconds

Request Parameters

Parameter	Type	Mandatory	Description
exchange	string	Yes	Allowed values: "EXCHANGE_2" (case insensitive)

Remember — Use valid API key and signature!

pythonjava

```
import requests
import json

params = {
    "exchange": "EXCHANGE_2"
}

payload = {}

endpoint = "/trade/api/v2/futures/all-pairs/ticker"

endpoint += ('&', '?')[urlparse(endpoint).query == ''] + urlencode(params)

url = "https://coinswitch.co" + endpoint

headers = {
    'Content-Type': 'application/json',
    'X-AUTH-SIGNATURE': <api-signature>, #https://api-trading.coinswitch.co/#
    'X-AUTH-APIKEY': <api-key>
}

try:
    response = requests.request("GET", url, headers=headers, json=params)
    print("Response JSON:", response.json())
except Exception as e:
    print(f"An error occurred: {e}")
```

The above command returns JSON structured like this:

```
STATUS CODE 200
{
  "data": {
    "1000FLOKIUSD": {
      "low_price_24h": "0.21655",
      "high_price_24h": "0.23397",
      "last_price": "0.22282",
      "symbol": "1000FLOKIUSD",
      "exchange": "EXCHANGE_2",
      "timestamp": 1732821806380,
      "best_ask_price": "0.22283",
      "best_bid_price": "0.22280",
      "price_24h_pcnt": "-1.192800",
      "base_asset_volume_24h": "195330555.0000",
      "quote_asset_volume_24h": "43903525.4670",
      "index_price": 0.22245,
      "mark_price": 0.2228,
      "open_interest": "270150998",
      "open_interest_value": "60189642.35",
      "funding_rate": 0.00041172,
      "next_funding_timestamp": 1732838400000,
      "best_bid_size": "313",
      "best_ask_size": "34"
    },
    "BTCUSD": {
      "low_price_24h": "94707.00",
      "high_price_24h": "97276.00",
      "last_price": "95136.60",
      "symbol": "BTCUSD",
      "exchange": "EXCHANGE_2",
      "timestamp": 1732821806680,
      "best_ask_price": "95136.70",
      "best_bid_price": "95136.60",
      "price_24h_pcnt": "-1.297300",
      "base_asset_volume_24h": "93707.6800",
      "quote_asset_volume_24h": "8970676685.2608",
      "index_price": 95046.53,
      "mark_price": 95136.7,
      "open_interest": "67529.878",
      "open_interest_value": "6424569744.32",
      "funding_rate": 0.00039681,
      "next_funding_timestamp": 1732838400000,
      "best_bid_size": "3.189",
      "best_ask_size": "3.963"
    },
    "ETHUSD": {
      "low_price_24h": "3531.12",
      "high_price_24h": "3689.79",
      "last_price": "3569.99",
      "symbol": "ETHUSD",
      "exchange": "EXCHANGE_2",
      "timestamp": 1732821805872,
      "best_ask_price": "3570.00",
      "best_bid_price": "3569.99",
      "price_24h_pcnt": "-0.853100",
      "base_asset_volume_24h": "1504238.4300",
      "quote_asset_volume_24h": "5427430066.1392",
      "index_price": 3566.07,
      "mark_price": 3569.66,
      "open_interest": "1165625.67",
      "open_interest_value": "4160887329.17",
      "funding_rate": 0.00044713,
      "next_funding_timestamp": 1732838400000,
      "best_bid_size": "32.33",
      "best_ask_size": "2.00"
    },
    "ADAUSD": {
      "low_price_24h": "0.9711",
      "high_price_24h": "1.0394",
      "last_price": "1.0015",
      "symbol": "ADAUSD",
      "exchange": "EXCHANGE_2",
      "timestamp": 1732821806104,
      "best_ask_price": "1.0015",
      "best_bid_price": "1.0014",
      "price_24h_pcnt": "-2.206800",
      "base_asset_volume_24h": "350765017.0000",
      "quote_asset_volume_24h": "352196711.5540",
      "index_price": 1.0003,
      "mark_price": 1.0012,
      "open_interest": "235312356",
      "open_interest_value": "235594730.83",
      "funding_rate": 0.00045597,
      "next_funding_timestamp": 1732838400000,
      "best_bid_size": "6027",
      "best_ask_size": "246"
    },
    "DOGEUSD": {
      "low_price_24h": "0.39143",
      "high_price_24h": "0.42943",
      "last_price": "0.39874",
      "symbol": "DOGEUSD",
      "exchange": "EXCHANGE_2",
      "timestamp": 1732821805604,
      "best_ask_price": "0.39875",
      "best_bid_price": "0.39874",
      "price_24h_pcnt": "-0.921800",
      "base_asset_volume_24h": "3806885381.0000",
      "quote_asset_volume_24h": "1543061421.3868",
      "index_price": 0.39839,
      "mark_price": 0.39871,
      "open_interest": "2554940115",
      "open_interest_value": "1018680173.25",
      "funding_rate": 0.00026608,
      "next_funding_timestamp": 1732838400000,
      "best_bid_size": "32177",
      "best_ask_size": "16740"
    },
    "1000PEPEUSD": {
      "low_price_24h": "0.0189620",
      "high_price_24h": "0.0203749",
      "last_price": "0.0195068",
      "symbol": "1000PEPEUSD",
      "exchange": "EXCHANGE_2",
      "timestamp": 1732821805704,
```

Futures WebSockets

How to connect ?

Socket Config	Value
Type	Socket.IO
Version	v4
Handshake Path	/pro/realtime-rates-socket/futures/{parameter}
Base URL	wss://ws.coinswitch.co/

Parameter	Type	Mandatory	Description
parameter	string	Yes	(case insensitive) Allowed values "exchange_2"

Namespace

Exchange	Namespace
EXCHANGE_2	baseurl + /exchange_2

Order Book

Whenever a new order is placed or a change happens in order book/market depth, web socket will send an update.

Event Name = "FETCH_ORDER_BOOK_CS_PRO"

Note :- This will give snapshot of the order_book at that timestamp

python java

```
"price_24h_pcmt": "-2.137600",
"base_asset_volume_24h": "35943473800.0000",
"quote_asset_volume_24h": "706593280.9413",
"index_price": 0.0194707,
"mark_price": 0.0194997,
"open_interest": "21465240900",
"open_interest_value": "418565757.98",
"funding_rate": 0.00084028,
"next_funding_timestamp": 1732838400000,
"best_bid_size": "41500",
"best_ask_size": "300"
},
"DASHUSDT": {
  "low_price_24h": "35.37",
  "high_price_24h": "38.75",
  "last_price": "36.65",
  "symbol": "DASHUSDT",
  "exchange": "EXCHANGE_2",
  "timestamp": 1732821806294,
  "best_ask_price": "36.67",
  "best_bid_price": "36.66",
  "price_24h_pcmt": "-2.318700",
  "base_asset_volume_24h": "109829.6100",
  "quote_asset_volume_24h": "4036827.9508",
  "index_price": 36.61,
  "mark_price": 36.65,
  "open_interest": "172245.68",
  "open_interest_value": "6312804.17",
  "funding_rate": 0.00025568,
  "next_funding_timestamp": 1732838400000,
  "best_bid_size": "31.50",
  "best_ask_size": "15.75"
},
"HMSTRUSDT": {
  "low_price_24h": "0.003727",
  "high_price_24h": "0.003968",
  "last_price": "0.003876",
  "symbol": "HMSTRUSDT",
  "exchange": "EXCHANGE_2",
  "timestamp": 1732821804869,
  "best_ask_price": "0.003876",
  "best_bid_price": "0.003875",
  "price_24h_pcmt": "0.025800",
  "base_asset_volume_24h": "4870172860.0000",
  "quote_asset_volume_24h": "18715274.1632",
  "index_price": 0.003871,
  "mark_price": 0.003875,
  "open_interest": "3444025580",
  "open_interest_value": "13345599.12",
  "funding_rate": 0.00038998,
  "next_funding_timestamp": 1732838400000,
  "best_bid_size": "350230",
  "best_ask_size": "167430"
},
"EIGENUSDT": {
  "low_price_24h": "3.5606",
  "high_price_24h": "4.1524",
  "last_price": "3.6003",
  "symbol": "EIGENUSDT",
  "exchange": "EXCHANGE_2",
  "timestamp": 1732821805369,
  "best_ask_price": "3.6007",
  "best_bid_price": "3.5999",
  "price_24h_pcmt": "-6.747300",
  "base_asset_volume_24h": "17627689.0000",
  "quote_asset_volume_24h": "67600321.6591",
  "index_price": 3.5964,
  "mark_price": 3.5991,
  "open_interest": "6218291",
  "open_interest_value": "22380251.14",
  "funding_rate": 0.0001,
  "next_funding_timestamp": 1732838400000,
  "best_bid_size": "217",
  "best_ask_size": "3"
}
}
}
```

PRO Tip: To connect with a particular exchange, we have to provide namespace as we use multiplexing.



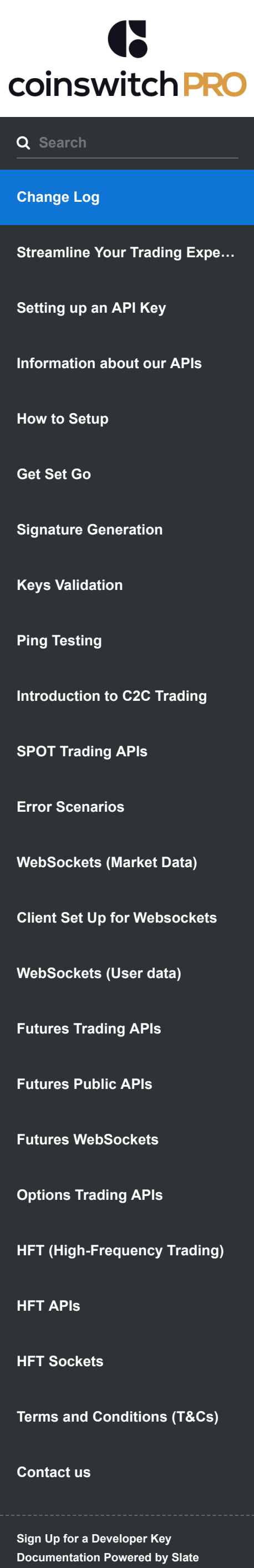
```
import socketio
sio = socketio.Client()

pair = "BTCUSDT"
sio.connect(url=base_url, namespaces=[namespace], transports='websocket',
            socketio_path='/pro/realtime-rates-socket/futures/exchange_2',
            subscribe_data = {
                'event': 'subscribe',
                'pair': pair
            })
sio.emit(FETCH_ORDER_BOOK_CS_PRO, subscribe_data, namespace=namespace)
```

RESPONSE



```
{
  "data": {
    "timestamp": 1732685131699,
    "asks": [
      [
        "3388.92", //Price
```

Ticker

The socket will share the ticker data last 1 min if trades are executed on given pair.

Event Name = "FETCH_TICKER_INFO_CS_PRO"

Trades

Whenever a trade gets executed, the web socket will send an update.

Event Name = "FETCH_TRADES_CS_PRO"

coinswitchPRO

Q Search

Change Log

Streamline Your Trading Expe...

Setting up an API Key

Information about our APIs

How to Setup

Get Set Go

Signature Generation

Keys Validation

Ping Testing

Introduction to C2C Trading

SPOT Trading APIs

Error Scenarios

WebSockets (Market Data)

Client Set Up for Websockets

WebSockets (User data)

Futures Trading APIs

Futures Public APIs

Futures WebSockets

Options Trading APIs

HFT (High-Frequency Trading)

HFT APIs

HFT Sockets

Terms and Conditions (T&Cs)

Contact us

Sign Up for a Developer Key

Documentation Powered by Slate

KLines

Event Name = "FETCH_CANDLESTICK_CS_PRO"

Note :- The pair(coin_pair) should be given in the following format
coinName_interval (eg: BTCUSDT_5, where BTCUSDT is the coin and 5 is the interval in which Klines are required) ** Interval is in minutes (eg: BTCUSDT_5, here 5 represents interval of 5 minutes)

Options Trading APIs

Options trading via API is currently live for a select group of users.

To request access, please follow these steps:

1. Email `api@coinswitch.co` from your Coinswitch-registered email ID
2. Use the subject line: "Request for Options API Trading Access"
3. Include your public API key in the email

Once received:

- Our team will whitelist your API key and confirm via email
- You'll receive a Python wrapper and the complete list of Options API endpoints

For any issues or support, feel free to reach out at `api@coinswitch.co` .

HFT (High-Frequency Trading)

Information about our APIs

The CoinSwitch HFT (High-Frequency Trading) APIs are designed specifically for high-volume traders who require ultra-low latency and higher rate limits compared to standard APIs. Currently these are available only for Futures and options trading. To request access, please follow these steps: Email `api@coinswitch.co` from your Coinswitch-registered email ID Use the subject line: "Request for HFT Trading API Access" Include your public API key in the email Once received: Our team will whitelist your API key and confirm via email You can Transfer funds using `/dma/api/v1/funds/transfer` API from your Coinswitch wallet to HFT wallet. Then set the account type, place order If you change the category name to options, you will be able to place options orders. By default, category name is linear through which you will be able to place futures trades For any issues or support, feel free to reach out at `api@coinswitch.co`.

SANDBOX ENDPOINTS

BASE_URL = "https://dms-uat.coinswitch.co/"

SOCKET_URL = "wss://pc-nats-uat.coinswitch.co"

PRODUCTION ENDPOINTS

BASE_URL = "https://dms.coinswitch.co/"

SOCKET_URL = "wss://pc-nats-prod.coinswitch.co"

In each of the APIs we need the following headers :

Header	Value
Content-Type	application/json
X-AUTH-SIGNATURE	Singature that you need to generate at your end
X-AUTH-APIKEY	API key provided by Coinswitch

HFT APIs

Get Server Time

python

java

```
python
{
    "E": 1732691683370,
    "p": 0.39378,
    "q": 32,
    "e": "EXCHANGE_2",
    "s": "DOGEUSDT",
    "m": true
},
{
    "E": 1732691672521,
    "p": 0.3941,
    "q": 2286,
    "e": "EXCHANGE_2",
    "s": "DOGEUSDT",
    "m": true
},
{
    "E": 1732691662133,
    "p": 0.39401,
    "q": 1211,
    "e": "EXCHANGE_2",
    "s": "DOGEUSDT",
    "m": true
}
]
}

import socketio
sio = socketio.Client()

pair = "BTCUSDT_5"
sio.connect(url=base_url, namespaces=[namespace], transports='websocket',
            socketio_path='/pro/realtime-rates-socket/futures/exchange_2', v
subscribe_data = {
    'event': 'subscribe',
    'pair': pair
}

sio.emit(FETCH_CANDLESTICK_CS_PRO, subscribe_data, namespace=namespace)

RESPONSE
{
  {
    "o": "104562.7", // open
    "h": "104588.5", // high
    "l": "104535",  // low
    "c": "104558.7", // close
    "v": "43.205",  // baseAsset volume
    "q": "1918381.4344",  // quoteAsset volume
    "s": "BTCUSDT", // symbol
    "i": "5", // interval
    "x": false, // isKline Closed
    "t": 1734343500000, // Kline start_time
    "T": 1734343799999, // Kline close_time
    "ts": 1734343543430 // time when last trade happens
  }
}
```

HFT Sockets



Q Search

Change Log

Streamline Your Trading Expe...

Setting up an API Key

Information about our APIs

How to Setup

Get Set Go

Signature Generation

Keys Validation

Ping Testing

Introduction to C2C Trading

SPOT Trading APIs

Error Scenarios

WebSockets (Market Data)

Client Set Up for Websockets

WebSockets (User data)

Futures Trading APIs

Futures Public APIs

Futures WebSockets

Options Trading APIs

HFT (High-Frequency Trading)

HFT APIs

HFT Sockets

Terms and Conditions (T&Cs)

Contact us

Sign Up for a Developer Key

Documentation Powered by Slate

Get server timestamp for synchronization

Request Parameters: None

Key Fields:

- timeSecond: Server time in seconds

- timeNano: Server time in nanoseconds

Get Instruments Info

Get contract specifications and trading rules

Key Trading Rules:

- Leverage: 1x to 100x (0.01 step)

- Min Order: 0.001 BTC (~\$109)

- Max Order: 1,190 BTC

- Tick Size: \$0.10

- Min Value: \$5

Get Tickers

Get real-time price and market statistics

Key Fields:

- lastPrice: Current market price (\$109,740)

- markPrice: Mark price for PnL calculation

- fundingRate: Current funding rate (-0.00000874%)

- volume24h: 24-hour volume (65,158 BTC)

- bid1Price/ask1Price: Best bid/ask prices

python

java

```
import requests
import json

params = {}

payload = {}

endpoint = "/v5/market/time"

endpoint += ('&', '?')[urlparse(endpoint).query == ''] + urlencode(params)

url = "http://dms-uat.coinswitch.co" + endpoint

headers = {
    'Content-Type': 'application/json',
    'X-AUTH-SIGNATURE': <api-signature>,
    'X-AUTH-APIKEY': <api-key>
}

try:
    response = requests.request("GET", url, headers=headers, json=params)
    print("Response JSON:", response.json())
except Exception as e:
    print(f"An error occurred: {e}")

STATUS_CODE 200
{
  "retCode": 0,
  "retMsg": "OK",
  "result": {
    "timeSecond": "1756460688",
    "timeNano": "1756460688504684927"
  },
  "retExtInfo": {},
  "time": 1756460688504,
  "settlement_currency_conversion_price": "94",
  "margin_currency_short_name": "INR"
}
```

```
import requests
import json

params = {
    "category": "linear",
    "symbol": "BTCUSDT",
    "limit":1
}

payload = {}

endpoint = "/v5/market/instruments-info"

endpoint += ('&', '?')[urlparse(endpoint).query == ''] + urlencode(params)

url = "http://dms-uat.coinswitch.co" + endpoint

headers = {
    'Content-Type': 'application/json',
    'X-AUTH-SIGNATURE': <api-signature>,
    'X-AUTH-APIKEY': <api-key>
}

try:
    response = requests.request("GET", url, headers=headers, json=params)
    print("Response JSON:", response.json())
except Exception as e:
    print(f"An error occurred: {e}")

{
  "retCode": 0,
  "retMsg": "OK",
  "result": {
    "category": "linear",
    "list": [
      {
        "symbol": "BTCUSDT",
        "contractType": "LinearPerpetual",
        "status": "Trading",
        "baseCoin": "BTC",
        "quoteCoin": "USDT",
        "leverageFilter": {
          "minLeverage": "1",
          "maxLeverage": "100.00",
          "leverageStep": "0.01"
        },
        "priceFilter": {
          "minPrice": "0.10",
          "maxPrice": "1999999.80",
          "tickSize": "0.10"
        },
        "lotSizeFilter": {
          "maxOrderQty": "1190.000",
          "minOrderQty": "0.001",
          "qtyStep": "0.001",
          "minNotionalValue": "5"
        },
        "fundingInterval": 480,
        "settleCoin": "USDT",
        "settlement_currency_conversion_price": "94",
        "margin_currency_short_name": "INR"
      }
    ]
  }
}
```

```
import requests
import json

params = {
    "category": "linear",
    "symbol": "BTCUSDT",
}

payload = {}

endpoint = "/v5/market/tickers"

endpoint += ('&', '?')[urlparse(endpoint).query == ''] + urlencode(params)

url = "http://dms-uat.coinswitch.co" + endpoint

headers = {
    'Content-Type': 'application/json',
    'X-AUTH-SIGNATURE': <api-signature>,
    'X-AUTH-APIKEY': <api-key>
}

try:
    response = requests.request("GET", url, headers=headers, json=params)
    print("Response JSON:", response.json())
except Exception as e:
```




coinswitchPRO

Q Search

Change Log

Streamline Your Trading Expe...

Setting up an API Key

Information about our APIs

How to Setup

Get Set Go

Signature Generation

Keys Validation

Ping Testing

Introduction to C2C Trading

SPOT Trading APIs

Error Scenarios

WebSockets (Market Data)

Client Set Up for Websockets

WebSockets (User data)

Futures Trading APIs

Futures Public APIs

Futures WebSockets

Options Trading APIs

HFT (High-Frequency Trading)

HFT APIs

HFT Sockets

Terms and Conditions (T&Cs)

Contact us

Sign Up for a Developer Key

Documentation Powered by Slate

Get Orderbook

Get live order book with bids and asks

Key Fields: - **b**: Bids array [price, quantity] - **a**: Asks array [price, quantity] - **ts**: Timestamp - **u**: Update ID - **seq**: Sequence number

Place Order

Place limit or market orders

Order Parameters: - **side**: "Buy" or "Sell" - **orderType**: "Limit", "Market" - **positionIdx**: 1 (Long), 2 (Short), 0 (One-way) - **timeInForce**: "GTC", "IOC", "FOK" - **orderLinkId**: Custom order ID (optional)

python

java

```
{
  "retCode": 0,
  "retMsg": "OK",
  "result": {
    "category": "linear",
    "list": [
      {
        "symbol": "BTCUSDT",
        "lastPrice": "109740.00",
        "indexPrice": "109794.46",
        "markPrice": "109738.20",
        "prevPrice24h": "112844.30",
        "price24hPcnt": "-0.027509",
        "highPrice24h": "113433.00",
        "lowPrice24h": "109346.70",
        "volume24h": "65158.1580",
        "turnover24h": "7276685854.7812",
        "fundingRate": "-0.00000874",
        "nextFundingTime": "1756483200000",
        "ask1Price": "109740.00",
        "ask1Size": "0.237",
        "bid1Price": "109739.90",
        "bid1Size": "0.302",
        "settlement_currency_conversion_price": "94",
        "margin_currency_short_name": "INR"
      }
    ]
  }
}
```

```
import requests
import json

params = {
  "category": "linear",
  "symbol": "BTCUSDT",
  "limit": 10
}

payload = {}

endpoint = "/v5/market/orderbook"

endpoint += ('&', '?')[urlparse(endpoint).query == ''] + urlencode(params)

url = "http://dms-uat.coinswitch.co" + endpoint

headers = {
  'Content-Type': 'application/json',
  'X-AUTH-SIGNATURE': <api-signature>,
  'X-AUTH-APIKEY': <api-key>
}

try:
  response = requests.request("GET", url, headers=headers, json=params)
  print("Response JSON:", response.json())
except Exception as e:
  print(f"An error occurred: {e}")
```

```
{
  "retCode": 0,
  "retMsg": "OK",
  "result": {
    "s": "BTCUSDT",
    "b": [
      ["109739.9", "8.007"],
      ["109739.8", "0.824"],
      ["109739.7", "0.12"],
      ["109739.6", "0.829"],
      ["109739.5", "0.001"]
    ],
    "a": [
      ["109740", "0.355"],
      ["109740.5", "0.001"],
      ["109742.6", "0.001"],
      ["109742.9", "0.001"],
      ["109743.5", "0.001"]
    ],
    "ts": 1756460692358,
    "u": 368327,
    "seq": 448248061021,
    "settlement_currency_conversion_price": "94",
    "margin_currency_short_name": "INR"
  }
}
```

```
import requests
import json

params = {}

payload = {
  "symbol": "BTCUSDT",
  "side": "Buy",
  "orderType": "Limit",
  "qty": "0.001",
  "price": "50000",
  "positionIdx": 1,
  "category": "linear",
  "timeInForce": "GTC",
  "orderLinkId": "custom_order_id_123"
}

endpoint = "/v5/order/create"

endpoint += ('&', '?')[urlparse(endpoint).query == ''] + urlencode(params)

url = "http://dms-uat.coinswitch.co" + endpoint

headers = {
  'Content-Type': 'application/json',
  'X-AUTH-SIGNATURE': <api-signature>,
  'X-AUTH-APIKEY': <api-key>
}

try:
  response = requests.request("POST", url, headers=headers, json=payload)
  print("Response JSON:", response.json())
except Exception as e:
  print(f"An error occurred: {e}")
```

```
{
  "retCode": 0,
  "retMsg": "OK",
  "result": {
    "orderId": "252b350d-ca18-4aca-9411-fcef79c122ec",
    "orderLinkId": "custom_order_id_123"
  },
  "retExtInfo": {},
  "time": 1756460699967,
```

coinswitchPRO

Q Search

Change Log

Streamline Your Trading Expe...

Setting up an API Key

Information about our APIs

How to Setup

Get Set Go

Signature Generation

Keys Validation

Ping Testing

Introduction to C2C Trading

SPOT Trading APIs

Error Scenarios

WebSockets (Market Data)

Client Set Up for Websockets

WebSockets (User data)

Futures Trading APIs

Futures Public APIs

Futures WebSockets

Options Trading APIs

HFT (High-Frequency Trading)

HFT APIs

HFT Sockets

Terms and Conditions (T&Cs)

Contact us

Sign Up for a Developer Key

Documentation Powered by Slate

Get Order Details Get detailed order information Order Status Values: - New: Order placed, not executed - PartiallyFilled: Partially executed - Filled: Fully executed - Cancelled: Cancelled by user - Rejected: Rejected by system	Get Trade History Get execution history for orders Note: Empty list means no executions (order not filled)
Switch Position Mode Switch between One-way and Hedge mode Position Modes: - 0: Merged Single (One-way mode) - 3: Both Sides (Hedge mode) ✓ Recommended	

python

java

```

}

import requests
import json

params = {
    "symbol": "BTCUSD",
    "orderLinkId": "custom_order_id_123",
    "category": "linear"
}

payload = {}

endpoint = "/v5/order/realtime"

endpoint += ('&', '?')[urlparse(endpoint).query == ''] + urlencode(params)

url = "http://dms-uat.coinswitch.co" + endpoint

headers = {
    'Content-Type': 'application/json',
    'X-AUTH-SIGNATURE': <api-signature>,
    'X-AUTH-APIKEY': <api-key>
}

try:
    response = requests.request("GET", url, headers=headers, json=params)
    print("Response JSON:", response.json())
except Exception as e:
    print(f"An error occurred: {e}")

{
    "retCode": 0,
    "retMsg": "OK",
    "result": {
        "category": "linear",
        "list": [
            {
                "symbol": "BTCUSD",
                "orderId": "252b350d-ca18-4aca-9411-fcef79c122ec",
                "orderLinkId": "custom_order_id_123",
                "orderType": "Limit",
                "orderStatus": "New",
                "side": "Buy",
                "qty": "0.001",
                "price": "50000",
                "timeInForce": "GTC",
                "leavesQty": "0.001",
                "cumExecQty": "0",
                "cumExecValue": "0",
                "cumExecFee": "0",
                "avgPrice": "",
                "lastPriceOnCreated": "109761.4",
                "positionIdx": 1,
                "createdTime": "1756460699966",
                "updatedTime": "1756460699969",
                "rejectReason": "EC_NoError"
            }
        ]
    },
    "settlement_currency_conversion_price": "94",
    "margin_currency_short_name": "INR"
}

import requests
import json

params = {
    "symbol": "BTCUSD",
    "orderLinkId": "custom_order_id_123",
    "category": "linear"
}

payload = {}

endpoint = "/v5/execution/list"

endpoint += ('&', '?')[urlparse(endpoint).query == ''] + urlencode(params)

url = "http://dms-uat.coinswitch.co" + endpoint

headers = {
    'Content-Type': 'application/json',
    'X-AUTH-SIGNATURE': <api-signature>,
    'X-AUTH-APIKEY': <api-key>
}

try:
    response = requests.request("GET", url, headers=headers, json=params)
    print("Response JSON:", response.json())
except Exception as e:
    print(f"An error occurred: {e}")

{
    "retCode": 0,
    "retMsg": "OK",
    "result": {
        "nextPageCursor": "",
        "category": "linear",
        "list": []
    },
    "settlement_currency_conversion_price": "94",
    "margin_currency_short_name": "INR"
}

import requests
import json

params = {}

payload = {
    "mode": 3,
    "coin": "USD",
    "category": "linear"
}

endpoint = "/v5/position/switch-mode"

endpoint += ('&', '?')[urlparse(endpoint).query == ''] + urlencode(params)

url = "http://dms-uat.coinswitch.co" + endpoint

headers = {
    'Content-Type': 'application/json',
    'X-AUTH-SIGNATURE': <api-signature>,

```



Q Search

Change Log

Streamline Your Trading Expe...

Setting up an API Key

Information about our APIs

How to Setup

Get Set Go

Signature Generation

Keys Validation

Ping Testing

Introduction to C2C Trading

SPOT Trading APIs

Error Scenarios

WebSockets (Market Data)

Client Set Up for Websockets

WebSockets (User data)

Futures Trading APIs

Futures Public APIs

Futures WebSockets

Options Trading APIs

HFT (High-Frequency Trading)

HFT APIs

HFT Sockets

Terms and Conditions (T&Cs)

Contact us

Sign Up for a Developer Key

Documentation Powered by Slate

Get Open Positions

Get current position information

Key Fields: - **side:** "Buy" (Long) or "Sell" (Short) - **size:** Position size (0.005 BTC) - **avgPrice:** Average entry price (\$109,729.42) - **unrealisedPnl:** Unrealized profit/loss (+\$0.13) - **leverage:** Current leverage (2x) - **positionIdx:** 1 (Long), 2 (Short)

Set Leverage

Set leverage for trading pair

Note: Error code 110043 means leverage is already at requested level

Transfer Funds

Transfer funds between accounts

Success Criteria: `"message": "transfer successful"`

Transfer Directions: - **"IN":** Transfer to futures account - **"OUT":** Transfer from futures account

python **java**

```
try:
    response = requests.request("GET", url, headers=headers, json=payload)
    print("Response JSON:", response.json())
except Exception as e:
    print(f"An error occurred: {e}")
```

```
{
  "retCode": 0,
  "retMsg": "All symbols switched successfully.",
  "result": {},
  "retExtInfo": {},
  "time": 1756460696928,
  "settlement_currency_conversion_price": "94",
  "margin_currency_short_name": "INR"
}
```

```
import requests
import json

params = {
    "symbol": "BTCUSDT",
    "category": "linear"
}

payload = {}

endpoint = "/v5/position/list"

endpoint += ('&', '?')[urlparse(endpoint).query == ''] + urlencode(params)

url = "http://dms-uat.coinswitch.co" + endpoint

headers = {
    'Content-Type': 'application/json',
    'X-AUTH-SIGNATURE': <api-signature>,
    'X-AUTH-APIKEY': <api-key>
}
```

```
try:
    response = requests.request("GET", url, headers=headers, json=params)
    print("Response JSON:", response.json())
except Exception as e:
    print(f"An error occurred: {e}")
```

```
{
  "retCode": 0,
  "retMsg": "OK",
  "result": {
    "category": "linear",
    "list": [
      {
        "symbol": "BTCUSDT",
        "side": "Buy",
        "size": "0.005",
        "positionValue": "548.6471",
        "avgPrice": "109729.42",
        "markPrice": "109755.4",
        "unrealisedPnl": "0.1299",
        "cumRealisedPnl": "-0.1920265",
        "leverage": "2",
        "positionIdx": 1,
        "positionStatus": "Normal",
        "tradeMode": 0,
        "autoAddMargin": 0,
        "positionIM": "274.47442796",
        "positionMM": "2.89411346",
        "createdTime": "1756456567526",
        "updatedTime": "1756460295495"
      }
    ]
  },
  "settlement_currency_conversion_price": "94",
  "margin_currency_short_name": "INR"
}
```

```
import requests
import json

params = {}

payload = {
    "symbol": "BTCUSDT",
    "buyLeverage": "2",
    "sellLeverage": "2",
    "category": "linear"
}

endpoint = "/v5/position/set-leverage"

endpoint += ('&', '?')[urlparse(endpoint).query == ''] + urlencode(params)

url = "http://dms-uat.coinswitch.co" + endpoint

headers = {
    'Content-Type': 'application/json',
    'X-AUTH-SIGNATURE': <api-signature>,
    'X-AUTH-APIKEY': <api-key>
}
```

```
try:
    response = requests.request("POST", url, headers=headers, json=payload)
    print("Response JSON:", response.json())
except Exception as e:
    print(f"An error occurred: {e}")
```

```
{
  "retCode": 110043,
  "retMsg": "leverage not modified",
  "result": {},
  "retExtInfo": {},
  "time": 1756460695432,
  "settlement_currency_conversion_price": "94",
  "margin_currency_short_name": "INR"
}
```

```
import requests
import json

params = {}

payload = {
    "direction": "IN",
    "amount": 0.01,
```


coinswitchPRO
Q Search
Change Log
Streamline Your Trading Expe...
Setting up an API Key
Information about our APIs
How to Setup
Get Set Go
Signature Generation
Keys Validation
Ping Testing
Introduction to C2C Trading
SPOT Trading APIs
Error Scenarios
WebSockets (Market Data)
Client Set Up for Websockets
WebSockets (User data)
Futures Trading APIs
Futures Public APIs
Futures WebSockets
Options Trading APIs
HFT (High-Frequency Trading)
HFT APIs
HFT Sockets
Terms and Conditions (T&Cs)
Contact us
Sign Up for a Developer Key Documentation Powered by Slate

Ticker

Use the following endpoint to get ticker data of an asset on the exchange:

EVENT_NAME = "v1.f.ex1.public.futures.ticker.>"

EVENT_NAME = "v1.f.ex1.public.futures.ticker.BTC/USDT"

Order-updates

Use the following endpoint to get order-updates:

EVENT_NAME = v1.f.ex1.private.subAccountId.491363048.order

python

java

```
]
],
"e": "COINSWITCHX",
"E": 1756475722084 //Exchange Timestamp
}
]
```

```
[
{
  "t": 0,
  "e": "COINSWITCHX", //Exchange
  "Epoch": 1756476914647,
  "ticker_data": [
    {
      "t": 1756476541682, //Server Timestamp
      "m": "BTC/USDT", // Market
      "o": "112762.00", // Open
      "h": "113276.40", //High
      "l": "108480.00", //Low
      "c": "108719.70", //Close
      "e": "COINSWITCHX", //Exchange
      "E": 1756476914647, //Epoch
      "bv": "83749.1230", //Base Asset Volume
      "qv": "9294150459.9087", //Quote Asset Volume
      "p": "-3.584800", // Percentage Change
      "b": "108719.60", // Best Bid Price
      "a": "108719.70", //Best Ask Price
      "T": 1756483200000, //Next Funding Timestamp
      "p": 108719, //Mark Price
      "i": 108754.05, //Index Price
      "s": 0, //Settle Price
      "r": 1.773e-05, //Funding Rate
      "oi": "67075.848", //Open Interest
      "oiv": "7292419118.71", //Open Interest Size
      "bs": "5.474", //Best Bid Size
      "as": "2.557" //Best Ask Size
    }
  ]
}
```

```
{
  "t": 0,
  "e": "COINSWITCHX", //Exchange
  "Epoch": 1756476914647,
  "ticker_data": [
    {
      "t": 1756476541682, //Server Timestamp
      "m": "BTC/USDT", // Market
      "o": "112762.00", // Open
      "h": "113276.40", //High
      "l": "108480.00", //Low
      "c": "108719.70", //Close
      "e": "COINSWITCHX", //Exchange
      "E": 1756476914647, //Epoch
      "bv": "83749.1230", //Base Asset Volume
      "qv": "9294150459.9087", //Quote Asset Volume
      "p": "-3.584800", // Percentage Change
      "b": "108719.60", // Best Bid Price
      "a": "108719.70", //Best Ask Price
      "T": 1756483200000, //Next Funding Timestamp
      "p": 108719, //Mark Price
      "i": 108754.05, //Index Price
      "s": 0, //Settle Price
      "r": 1.773e-05, //Funding Rate
      "oi": "67075.848", //Open Interest
      "oiv": "7292419118.71", //Open Interest Size
      "bs": "5.474", //Best Bid Size
      "as": "2.557" //Best Ask Size
    }
  ]
}
```

Trade-updates

Use the following endpoint to get trade-updates:

EVENT_NAME = v1.f.ex1.private.subAccountId.491363048.trade

```json

Trade-Updates:

```
{
 "t": 1756478116649,
 "e": "COINSWITCHX",
 "event_type": "execution.linear",
 "E": 1756478116649,
 "T": 1756478116611,
 "User_id": "491363048",
 "sub_account_id": "491363048",
 "o":
 [
 {
 "category": "linear",
 "symbol": "BTCUSDT",
 "isLeverage": "0",
 "orderId": "316ea49e-59b3-42c5-9d71-edaf2504b0f8",
 "orderLinkId": "cf8cc52d-b749-4d97-aeb9-f44c90b94092",
 "side": "Sell",
 "orderPrice": "107396.1",
 "orderQty": "0.002",
 "leavesQty": "0",
 "createType": "CreateByUser",
 "orderType": "Market",
 "stopOrderType": "UNKNOWN",
 "execFee": "0.07593348",
 "execId": "a1076552-0c0f-5a56-a064-5a653f9172c6",
 "execPrice": "108476.4",
 "indexPrice": "",
 "markPrice": "",
 "underlyingPrice": "",
 "tradeIv": "",
 "markIv": "",
 "execQty": "0.002",
 "execType": "Trade",
 "execValue": "216.9528",
 "execTime": "1756478116611",
 "isMaker": false,
 "feeRate": "0.00035",
 "blockTradeId": "",
 "closedSize": "0.002",
 "seq": 448368491576
 }
]
}
```

Order-Updates Response:

```
{
 "t": 1756478182146,
 "e": "COINSWITCHX",
 "event_type": "order.linear",
 "E": 1756478182146,
 "T": 0,
```

coinswitchPRO

Q

Search

Change Log

Streamline Your Trading Expe...

Setting up an API Key

Information about our APIs

How to Setup

Get Set Go

Signature Generation

Keys Validation

Ping Testing

Introduction to C2C Trading

SPOT Trading APIs

Error Scenarios

WebSockets (Market Data)

Client Set Up for Websockets

WebSockets (User data)

Futures Trading APIs

Futures Public APIs

Futures WebSockets

Options Trading APIs

HFT (High-Frequency Trading)

HFT APIs

HFT Sockets

Terms and Conditions (T&Cs)

Contact us

Sign Up for a Developer Key

Documentation Powered by Slate

Terms and Conditions (T&Cs)

CoinSwitch Crypto Aggregator Platform

CoinSwitch PRO API Trading - Terms & Conditions

1. These Application Programming Interface (API) License Terms and Conditions (“**API Terms**”) shall govern the use of the API of CoinSwitch for the users of CoinSwitch PRO to undertake various trades (“**Services**”) by Users who fulfill the Eligibility Criteria stated in the [Terms of Use](#) (the “**Terms**”) and these API Terms. The User hereby agrees and acknowledges that upon accessing any CoinSwitch PRO API (*defined hereinafter*), and/or any other information, service, feature governed by terms contained herein, the User shall be bound by the Terms and these API Terms, as updated, modified, and/or replaced from time to time at the sole discretion of CoinSwitch without any prior notice to You. You are advised to review these API Terms for any such amendment, replacement or changes to the terms contained herein and any future use or access to any *Services* covered herein. These API Terms incorporate by reference and supplements the Terms and the [Trading Policy](#).

Capitalized terms used but not defined herein shall have the meanings given to them in the Terms and/or the Trading Policy. In the event of a conflict between the Terms and these API Terms or the Trading Policy and these API Terms, these API Terms will prevail to the extent of the conflict.

Contact us

You can reach out to us at [api@coinswitch.co](mailto:api@coinswitch.co) in case of any queries/suggestions

python

java

```
"o": {
 "category": "linear",
 "symbol": "SOLUSD",
 "orderId": "68a4579e-c396-4427-b055-7a8bcabfd48d",
 "orderLinkId": "7b708608-7e1d-456b-aa78-43a9892c2fda",
 "blockTradeId": "",
 "side": "Buy",
 "positionIdx": 0,
 "orderStatus": "Filled",
 "cancelType": "UNKNOWN",
 "rejectReason": "EC_NoError",
 "timeInForce": "IOC",
 "isLeverage": "",
 "price": "220.25",
 "qty": "0.2",
 "avgPrice": "209.76",
 "leavesQty": "0",
 "leavesValue": "0",
 "cumExecQty": "0.2",
 "cumExecValue": "41.952",
 "cumExecFee": "0.0146832",
 "orderType": "Market",
 "stopOrderType": "",
 "orderIv": "",
 "triggerPrice": "",
 "takeProfit": "",
 "stopLoss": "",
 "triggerBy": "",
 "tpTriggerBy": "",
 "slTriggerBy": "",
 "triggerDirection": 0,
 "placeType": "",
 "lastPriceOnCreated": "209.75",
 "closeOnTrigger": false,
 "reduceOnly": false,
 "smpGroup": 0,
 "smpType": "None",
 "smpOrderId": "",
 "slLimitPrice": "0",
 "tpLimitPrice": "0",
 "tpslMode": "UNKNOWN",
 "createType": "CreateByUser",
 "marketUnit": "",
 "createdTime": "CreateByUser",
 "updatedTime": "1756478182110",
 "feeCurrency": ""
}
```